recon

User Manual

Version 0.85.1

2026-05-24

Repository · https://github.com/codedeviate/recon
License · MIT

About this manual

recon is a versatile network reconnaissance CLI written in Rust. It started as a curl clone and grew into a multi-protocol investigation tool covering HTTP(S), TLS certificate inspection, DNS, WHOIS, ping, traceroute, barcode encoding and decoding, file compression and archiving, markdown/HTML/PDF conversion, and a full Rhai script engine that exposes every protocol probe and helper for automation.

This manual covers every user-facing flag, every script binding, and shows how to combine them. The built-in `recon --help <topic>` command remains the quick reference; this document is the long-form companion.

A PDF rendering of this manual lives alongside it at `MANUAL.pdf`. It is regenerated every time the markdown changes — see `CLAUDE.md` for the maintenance policy. Generate locally with:

```
recon --md-to-pdf docs/MANUAL.md \
    --toc --toc-depth 3 --gfm --unsafe-html --page-break-on-h1 \
    --doc-title 'recon User Manual' \
    --doc-author 'Thomas Bjork' \
    --doc-subject 'recon CLI reference and script-engine guide' \
    --doc-keywords 'recon, curl, network, reconnaissance, scripting' \
    -o docs/MANUAL.pdf
```

Table of Contents

Contents

The table of contents above is auto-generated from H1/H2/H3 headings. Every entry is a clickable anchor in the PDF. For a narrative walkthrough of the structure, the four parts are:

- **Part I — Getting started**: introduction, installation, quick start.
- **Part II — CLI reference**: every flag grouped by area, with examples.
- **Part III — Script engine**: the Rhai interpreter, every binding, many examples.
- **Part IV — Appendices**: exit codes, env vars, config, glossary.

Introduction

recon is a single-binary command-line tool. Its design goals, in rough order:

1. **Curl-compatible** where it overlaps with curl. Flag names, short forms, and behaviours match curl for HTTP(S) requests so existing tooling keeps working. `recon --help curl` lists the compatibility status.
2. **Pure Rust** where possible. reqwest + rustls for HTTP(S); hickory for DNS; comrak for markdown; flate2/brotli/zstd for compression; rxing for barcodes; tungstenite for WebSockets; rumqttc for MQTT. Where a pure-Rust path doesn't exist (headless-Chrome PDF rendering, for instance), recon shells out to a named external CLI (`agent-browser`) rather than linking the external dependency.
3. **Protocol-rich**. HTTP(S), FTP(S), SFTP, TFTP, Gopher, SMTP(S), POP3(S), IMAP(S), SSH, SCP, Telnet, LDAP(S), WS(S), RTSP(S), Dict, NTP, Memcached, Redis, MQTT(S), IPFS/IPNS, Unix sockets, plus ping/traceroute.
4. **Scriptable**. A Rhai script engine exposes every protocol probe, plus cryptography, compression, barcodes, JSON/YAML/XML handling, SQLite, file I/O, and multi-threaded concurrency. Scripts can stand in for shell pipelines and integration-test harnesses.
5. **Diagnostic-friendly**. Verbose output, curl-style write-out, prettified JSON/XML, precise error messages, exit codes that match curl's.

Installation

From source

```
git clone https://github.com/codedeviate/recon
cd recon
cargo build --release
cp target/release/recon ~/.local/bin/     # or /usr/local/bin with sudo
```

You'll need:

- Rust 1.75 or newer (`rustup install stable`).
- A system linker; `cargo` handles the rest.
- For the optional `--html-to-pdf` / `--md-to-pdf` features: `agent-browser` on `$PATH` (`brew install agent-browser` on macOS, or `npm install -g agent-browser` elsewhere).

First-run bootstrap

```
recon --init
```

creates `~/.recon/` with `script/`, `jars/`, `sni/`, and a commented `config.toml` skeleton. Existing files are never overwritten. See the `~/.recon/` layout appendix.

Quick start

```
# HTTP(S)
recon https://api.example.com/status
recon -X POST https://api.example.com/users -d '{"name":"a"}' -H 'Content-
Type: application/json'
recon --json '{"q":"rust"}' https://api.example.com/search        # auto-
sets headers
recon -L https://short.url/foo                                    # follow
redirects
recon https://site.example.com --LHEAD                            # follow
+ show each hop

# Inspection
recon https://example.com --cert                                  # inspect
TLS cert
recon example.com --dns                                           #
A/AAAA/MX/TXT/NS/CNAME/SOA
recon --ping 8.8.8.8                                              # TCP +
ICMP ping
recon --traceroute 8.8.8.8
recon --whois example.com

# Email-protection
recon example.com --spf --dmarc --dkim selector1 --mta-sts --bimi --tls-rpt

# Barcodes
recon --encode qr -o out.png 'https://example.com'
recon --decode out.png                                            # →
qr<TAB>https://example.com

# Conversions
recon --md-to-pdf README.md --toc --gfm -o README.pdf
recon --compare a.json b.json                                     # GNU-
diff compatible

# Scripting
recon --script my-probe.rhai https://api.example.com
recon --init                                                      #
bootstrap ~/.recon/

# Help surface
recon --help                                                      # clap-
generated flag list
recon --flags                                                     # curl-
style alphabetical index
recon --help http                                                 # deep-
dive on HTTP topic
recon --help script                                               # deep-
dive on scripting
```

```
recon --examples                                                 # curated
examples, paged
recon --version                                                  #
protocols + features
```

How recon is organized

recon has one binary with many modes. The mode is selected by flag:

Mode group	Selector flags
HTTP request (default)	no mode flag, or a URL positional argument
Source-inspection	`--hash`, `--compress`, `--decompress`, `--encode`, `--decode`, `--encrypt`, `--decrypt`
Email-protection	`--spf`, `--dmarc`, `--dkim`, `--mta-sts`, `--bimi`, `--tls-rpt`
Network tests	`--ping`, `--traceroute`, `--netstatus`, `--whois`
Certificate inspection	`--cert`, `--dns`, or positional URL with `--cert` / `--dns`
SMTP send	`--mail-from` and `--mail-to`
Serve	`--serve`, `--serve-tls`
JWT	`--jwt-view`, `--jwt-sign`, `--jwt-validate`
Archive	`--archive`, `--extract`
Checkdigit	`--checkdigit`, `--checkdigit-create`, `--checkdigit-list`
Sample	`--sample`, `--sample-list`
Browser	`--browser-screenshot`
Compare	`--compare`
Docs	`--md-to-html`, `--md-to-pdf`, `--html-to-pdf`
Script	`--script` (turns recon into a Rhai interpreter)
Meta	`--init`, `--editor-cleanup`, `--list-charsets`, `--iconv`

Only the modes in the first group do an HTTP request. Every other mode is stand-alone.

Global conventions

- **`-h`, `--help`** prints the clap-generated flag summary. `--help <topic>` routes to a deep-dive help topic (see `recon --help` footer for the list).
- **`-V`, `--version`** prints the curl-compatible multi-line banner. `--version-short` prints just the version number.
- **`-v`** increases verbosity. Repeat (`-vv`, `-vvv`) for more detail. At default, connection lines and protocol handshakes are suppressed.
- **`-s`, `--silent`** suppresses progress + verbose output. `--show-error` re-enables error output when `-s` is set.
- **`-o <FILE>`** writes the response body to a file instead of stdout. `-O` / `--remote-name` derives the filename from the URL's path. `-J` / `--remote-header-name` respects Content-Disposition.
- **`-f`, `--fail`** exits non-zero on HTTP 4xx/5xx (no body printed). `--fail-with-body` prints the body but still exits non-zero.
- **`-k`, `--insecure`** disables TLS cert verification. Use for self-signed tests; never in production scripts.
- **`-p`, `--prettify`** pretty-prints JSON / XML / YAML bodies. Auto-detection by Content-Type; can be disabled with `--no-prettify`.
- **`-L`, `--location`** follows HTTP redirects. `--max-redirs N` caps the chain (default 10). `--LHEAD` follows and prints each hop's headers.
- **`-A "..."`** sets the User-Agent. Default is `recon/<version>`.
- **`-e <URL>`** sets the Referer header.
- **`-H 'Name: Value'`** adds a request header. Repeat for multiple.
- **`--max-time <SECS>`** overall operation timeout. **`--connect-timeout <SECS>`** TCP connect timeout (default 30).
- **`-w '<FORMAT>'`** writes a curl-compatible write-out string to stdout after the body. See Write-out format.
- **`--json <DATA>`** curl's `--json` compatibility: sends DATA as a JSON body and auto-sets `Content-Type: application/json` + `Accept: application/json`.
- **`-T <FILE>`** uploads the file as the request body (default method PUT).
- **Environment variables** — many flags honor the same env var curl would (`HTTP_PROXY`, `HTTPS_PROXY`, `NO_PROXY`, `ALL_PROXY`, `SSL_CERT_FILE`, `CURL_CA_BUNDLE`). recon-specific ones are prefixed `RECON_`. See Environment variables.

Part II — CLI reference

HTTP / HTTPS requests

The default mode. A positional URL (or `--url <URL>`) triggers an HTTP request through the reqwest + rustls stack.

> **0.62.0 curl-parity additions** — see the curl easy-wins section of `recon --examples` and the relevant flags table rows below. Quick summary: `-r/--range`, `-z/--time-cond`, `--etag-compare`, `--etag-save`, `--timestamping`, `--max-filesize`, `--url-query`, `--disallow-username-in-url`, `--remove-on-error`, `--no-clobber`, `--create-file-mode`, `-N/--no-buffer`, `-D/--dump-header`, `--stderr`, `--no-progress-meter`, `--tcp-nodelay`, `--no-keepalive`, `--keepalive-time`, `--capath`, `--ca-native`, `--tls-max`, `--connect-to`, `--oauth2-bearer`, `--xattr`, `--spider`.

Core flags

Flag	Description
`-X, --request <METHOD>`	GET, POST, PUT, PATCH, DELETE, HEAD. Default GET (PUT when `-T` set, POST when `-d` set).
`-H, --header <NAME: VALUE>`	Add a request header. Repeatable.
`-d, --data <BODY | @FILE>`	Request body. `@file` reads from disk, `@-` reads stdin. Promotes GET → POST unless `-G`.
`--json <DATA>`	Curl-compatible: sends DATA as JSON, sets `Content-Type: application/json` + `Accept`.
`--data-raw <DATA>`	Like `-d` but `@file` is NOT processed (sends literal string).
`--data-binary <DATA>`	Like `-d` but CR/LF are not stripped from @file content.
`--data-urlencode <DATA>`	URL-encode DATA. Repeatable; values joined with `&`.
`-G, --get`	Send `-d` data as query parameters on GET (body empty).
`-T, --upload-file <PATH>`	Upload file as request body. Default PUT.
`-L, --location`	Follow redirects (3xx).

Flag	Description
`--max-redirs <N>`	Cap redirect chain (default 10).
`--LHEAD`	Follow redirects and print each hop's headers.
`-e, --referer <URL>`	Set Referer header. `--referrer` alias accepted.
`-A, --user-agent <STR>`	User-Agent. Default `recon/<version>`.
`--compressed`	Request + auto-decompress gzip/deflate/brotli/zstd.
`--max-time <SECS>`	Total operation timeout (seconds, fractional OK).
`--connect-timeout <SECS>`	TCP connect timeout (default 30).
`-f, --fail`	Exit non-zero on HTTP 4xx/5xx, suppress body.
`--fail-with-body`	Exit non-zero on 4xx/5xx but still print body.
`-4, --ipv4` / `-6, --ipv6`	Force IPv4 / IPv6 resolution.
`--resolve <HOST:PORT:ADDR>`	Override DNS for a specific host:port → addr. Repeatable.

Examples

```
# Simple requests
recon https://httpbin.org/get
recon -X POST https://api.example.com/v1/users -d '{"name":"a"}' -H 'Content-Type: application/json'
recon --json '{"query":"rust"}' https://api.example.com/search

# Body from files / stdin
recon https://upload.example.com -d @payload.json -H 'Content-Type: application/json'
cat body.json | recon -X POST https://api.example.com -d @-

# Upload
recon -T ./report.pdf https://upload.example.com/reports/       # PUT by default
recon -T ./report.pdf https://upload.example.com/ -X POST       # override method

# Follow redirects
recon -L https://bitly.example/abc
recon --LHEAD https://example.com/                              # show each hop's headers

# Fail fast
recon -f https://api.example.com/users/42                       # exit 22 on 4xx/5xx, no body
recon --fail-with-body https://api.example.com/users/42         # same but still print body

# Rate and timeout
recon --max-time 10 https://slow.example.com/
recon --connect-timeout 3 --max-time 8 https://flaky.example.com/
recon --limit-rate 500K https://download.example.com/big.bin -o big.bin
recon --speed-limit 10000 --speed-time 30 https://stalling.example.com/big.bin -O
```

Wget-style batch fetching

Long-form wget-compatible flags for batch URL handling. Short forms (`-A`, `-R`, `-w`, `-t`, `-r`, `-l`, `-m`, `-p`, `-k`, `-D`, `-H`, `-np`) are intentionally not provided — recon reserves single-letter flags for curl compatibility. The recursive/mirror cluster (`-r`, `-l`, `-m`, `-p`, `-k`, etc.) is deferred and tracked in `OUT-OF-SCOPE.md`.

Flag	Description
`--input-file <FILE>`	Batch-fetch URLs listed in FILE (one per line, `#` comments, blank lines ignored, `-` reads from stdin). Each URL processed

Flag	Description
	independently.
`--wait <SECS>`	(0.67.0) Fixed-seconds delay between URLs in batch mode. Skipped before the first URL. Overrides `--rate` when both are set.
`--tries <N>`	(0.67.0) Total attempts per URL (wget semantics: `tries = retries + 1`). Overrides `--retry`. `--tries 1` disables retries; `--tries 0` is rejected.
`--accept <LIST>`	(0.67.0) Comma-separated filename-suffix accept list (case-insensitive). e.g. `--accept jpg,png` keeps only URLs whose final path segment ends in `.jpg` or `.png`. URLs with empty final segments fail.
`--reject <LIST>`	(0.67.0) Comma-separated filename-suffix reject list. Combines with `--accept` (URL must pass both). URLs with empty final segments pass.
`--continue`	Resume an interrupted download (wget-style auto-detect from local file size). Equivalent to `--continue-at -`.
`--continue-at <OFFSET>`	Resume from BYTE offset (curl-compatible). `-` auto-detects.
`--spider`	HEAD-only check; print `<status> <url>` per URL and exit non-zero on any 4xx/5xx. Pairs with `--input-file`.
`--timestamping`	Skip download when local file's mtime ≥ server's `Last-Modified`. Sets `If-Modified-Since`.
`--rate <N/s|N/m|N/h>`	Request rate cap. Engages with `--input-file`: at most N requests per second/minute/hour. Overridden by `--wait`.

Examples

```
# Polite batch fetch with a 2-second gap between URLs
recon --input-file urls.txt --wait 2

# Filter to image URLs, drop thumbnails
recon --input-file urls.txt --accept jpg,png --reject thumb

# Wget-style retries (5 total attempts)
recon https://api.example.com/ --tries 5

# Spider check filtered by extension
recon --input-file urls.txt --spider --accept html,htm

# Resume an interrupted download
recon https://example.com/big.iso -o big.iso --continue
```

Output control

Flag	Description
`-o, --output <FILE>`	Write body to FILE (otherwise stdout).
`-O, --remote-name`	Filename = URL's last path segment (percent-decoded).
`--remote-name-all`	Apply `-O` to every URL in `--input-file`. Curl-parity for batch downloads.
`-J, --remote-header-name`	Use Content-Disposition filename when saving.
`--create-dirs`	Create parent dirs for `-o` path.
`-i, --include`	Print response headers before body.
`-I, --head`	Print headers only; no body. Implies `-X HEAD`.
`-s, --silent`	Suppress progress + verbose output.
`--show-error`	Re-enable error output even when `-s` is set.
`-v, --verbose`	Verbose. Repeatable: `-v`, `-vv`, `-vvv`.
`--progress`	Show a progress meter when saving to a file (opt-in).
`-#, --progress-bar`	`#`-character progress bar style (curl `-#` parity). Also activates the progress meter.

Flag	Description
`-p, --prettify`	Pretty-print JSON / XML / YAML bodies.
`--prettify-as <FORMAT>`	Force prettify format (json/xml/html/yaml/csv/tsv/auto). Implies `-p`. Use when auto-detect picks the wrong format or there is no Content-Type.
`--stdin`	Read body from stdin instead of making an HTTP request. Runs the post-fetch pipeline (prettify, `--output-charset`, `-o`) over the piped input. Mutually exclusive with a URL.
`--from-clipboard`	Read body from system clipboard (no HTTP request). Mutex with `--stdin` and URL.
`--to-clipboard`	Write output to system clipboard. Mutex with `-o` and `--editor`. UTF-8 text only.
`--clipboard [<DIR>]`	Use clipboard for I/O. `DIR` = `in` / `out` / `both`. Bare form auto-resolves direction from context.
`--no-prettify`	Disable auto-pretty (even when content-type suggests it).
`-w, --write-out <FORMAT>`	Print a curl-compatible summary after the body. See Write-out format.
`--editor [<CMD>]`	Open response body in `$EDITOR` after the request. URL-shaped next token (contains `://`) is treated as the positional URL — `recon --editor https://example.com` works without the `=` form.
`--json` (output-side inferred)	Combined with `-p`, forces JSON pretty-printing regardless of content-type.

Examples

```
# Save to file
recon https://example.com/favicon.ico -O                      # →
favicon.ico
recon https://example.com/api/users.json -o users.json
recon https://x.example.com/deep/path/file.txt --create-dirs -o
downloads/x/file.txt

# Batch download — save every URL to its own file (--remote-name-all,
0.73.0)
recon --input-file urls.txt --remote-name-all
recon --input-file urls.txt --remote-name-all --output-dir ./downloads/ --
rate 2/s

# Hash-style progress bar (curl -#, 0.73.0)
recon https://example.com/big.zip -O -#                        # # bar
instead of default
recon --input-file urls.txt --remote-name-all -#               # combined
with batch

# Inspect headers
recon -I https://example.com/                                  # HEAD
recon -i https://api.example.com/v1/status                     # headers +
body
recon -i -I https://example.com/                               # forced
HEAD with body suppressed

# Pretty-print
recon https://api.example.com/large.json -p
recon https://api.example.com/feed.xml -p                      # XML also
supported
curl -s https://api.example.com/blob.yaml | recon -p           # stdin
auto-detected

# --stdin: prettify a payload without making any HTTP request
recon --stdin -p                                   # auto-detect format from
body
recon --stdin --prettify-as json                   # force JSON
recon --stdin -p --prettify-as xml -o pretty.xml < raw.xml   # write to
file

# Auto-detected stdin: --stdin is optional when piping
echo '{"a":1}' | recon -p
cat data.json | recon --prettify-as json -o pretty.json

# Clipboard I/O — native clipboard read/write without shell pipes
recon --clipboard --prettify-as json                   # read from
clipboard, auto-resolves as input
recon --clipboard both --prettify-as json              # prettify in place
```

```
(read from, write back to)
recon https://api.example.com/data --to-clipboard     # fetch URL, copy
result to clipboard
recon --from-clipboard --editor vim                   # open clipboard body
in editor

# --prettify-as: force the format on a real HTTP response
recon https://api.example.com/data --prettify-as json   # implies -p
recon https://example.com/feed --prettify-as xml        # for servers that
lie about Content-Type

# Verbose progression
recon -v https://api.example.com/          # connection + TLS + headers
recon -vv https://api.example.com/         # plus intermediate reqwest info
recon -vvv https://api.example.com/        # plus raw handshake + wire dump

# Writeout
recon https://api.example.com/ -w '\n%{http_code} in %{time_total}s (%
{size_download} bytes)\n'
```

Authentication & TLS

Flag	Description
`-u, --user <USER:PASS>`	HTTP Basic credentials.
`-k, --insecure`	Skip TLS cert verification.
`--tlsv1.2` / `--tlsv1.3`	Force minimum TLS version.
`--cacert <PATH>`	Trust an extra PEM root (on top of system roots).
`--capath <DIR>`	Directory of `.pem` / `.crt` / `.cer` root certificates; each file is added as a trust root.
`--ca-native`	Disable built-in webpki roots; use the OS native trust store only.
`--crlfile <PATH>`	PEM file of X.509 CRLs. Server certs in any loaded CRL are rejected at handshake. Multi-CRL bundles supported.
`--interface <IP|NAME>`	Bind outgoing socket to a specific local IP **or** interface name (eth0, en0 on Linux/macOS via getifaddrs; Windows accepts IP literals only).
`--limit-rate <RATE>`	Throttle download. Suffixes: K, M, G.

Flag	Description
`--speed-limit <BYTES>`	Minimum bytes-per-second (fails if rate drops).
`--speed-time <SECS>`	Window for `--speed-limit` (default 30).

Examples

```
recon -u alice:s3cr3t https://private.example.com/
recon -k https://self-signed.example.com/
recon --tlsv1.3 https://example.com/                # refuse downgrade to 1.2
recon --cacert /etc/corp-root.pem https://internal.corp/
recon --capath /etc/pki/ca-trust/source/ https://internal.corp/
recon --ca-native https://example.com/              # OS trust store only
recon --crlfile /etc/pki/tls/crls/all.pem https://example.com/  # reject revoked certs
recon --interface 10.0.0.5 https://example.com/     # use a specific source IP
```

Client certificates (mTLS)

Shipped in 0.54.0. Present a client certificate during the TLS handshake for mutual TLS.

Flag	Description
`-E, --client-cert <PATH>`	PEM-encoded client cert. May include the key inline.
`--client-key <PATH>`	PEM key (when cert is cert-only).
`--cert-type <PEM|DER>`	Cert format. DER errors with a conversion recipe.
`--key-type <PEM|DER|ENG>`	Key format. ENG refused (rustls has no engine concept).
`--pass <PASS>`	Placeholder for encrypted PKCS#8 passphrase. Encrypted keys refused with an `openssl pkcs8` recipe.

Examples

```
# Combined cert+key PEM
recon -E ~/keys/bundle.pem https://mtls.example.com/

# Split cert and key
recon -E client.crt --client-key client.key https://mtls.example.com/

# Decrypt encrypted PKCS#8 externally first
openssl pkcs8 -in client.key.enc -out client.key
recon -E client.crt --client-key client.key https://mtls.example.com/
```

See also: `recon --help client-cert`.

Browser fingerprint impersonation (0.77.0, opt-in)

Shipped in 0.77.0 behind the `impersonate` Cargo feature. Routes the request through `wreq` (BoringSSL) + `wreq-util` to mimic a real browser's TLS ClientHello and HTTP/2 SETTINGS frame. Useful when a server uses JA3/JA4 fingerprinting or H2-frame analysis to distinguish bots from real browsers. (`wreq` is the renamed successor to `rquest`, which was yanked from crates.io after the upstream rename; recon-cli migrated in 0.80.7.)

The default `recon` binary is built without this feature so the binary stays small and skips the BoringSSL build dependency. Build with `cargo build --features impersonate`, or download the `recon-impersonate` release artifact.

Flag	Description
`--impersonate <PROFILE>`	Forwards to `wreq_util::Emulation`. Examples: `chrome_131`, `firefox_128`, `safari_17.5`, `edge_131`, `okhttp_5`, `chrome_android_131`, `safari_ios_17.4.1`. Hyphens accepted as a convenience (`chrome-131` ≡ `chrome_131`). See `recon --help impersonate` for the full list of supported profiles.
`--ja3 <STRING>`	**Deferred.** Reserved in the CLI for forward-compatibility; errors at runtime as not-yet-implemented. Use `--impersonate` for now. See OUT-OF-SCOPE.md for the upstream-blockers rationale.
`--ja4 <STRING>`	**Deferred.** Same.
`--http2-fingerprint <STRING>`	**Deferred.** Same.

V1 incompatibility list — these flags cannot combine with `--impersonate`: `--ciphers`, `--tls13-ciphers`, `--tlsv1.2`, `--tlsv1.3`, `--client-cert`, `--client-key`, `--cacert`, `--capath`. Reason: the impersonation profile owns the TLS configuration; user-supplied overrides would defeat the fingerprint.

Examples

```
# Build with the impersonate feature
cargo build --release --features impersonate

# Impersonate Chrome 131 against an HTTPS endpoint
recon --impersonate chrome_131 https://httpbin.org/headers

# Hyphens also work
recon --impersonate chrome-131 https://example.com/

# Verify the live fingerprint against a JA3 / JA4 echo server
recon --impersonate firefox_128 https://tls.peet.ws/api/all

# Mobile and OkHttp profiles
recon --impersonate chrome_android_131 https://example.com/
recon --impersonate safari_ios_17.4.1 https://example.com/
recon --impersonate okhttp_5 https://example.com/
```

The default (rustls-only) build rejects the four flags with a clear "rebuild with --features impersonate" hint pointing at the `recon-impersonate` release artifact.

See also: `recon --help impersonate`.

Proxy routing

0.50.0 shipped the full proxy suite. Schemes auto-detected from the URL: `http://`, `https://`, `socks5://`, `socks5h://`.

Flag	Description
`-x, --proxy <URL>`	Proxy URL. Falls back to `$HTTPS_PROXY` / `$HTTP_PROXY` / `$ALL_PROXY`.
`-U, --proxy-user <USER:PASS>`	Basic auth for the proxy.
`--noproxy <LIST>`	Comma-separated bypass list (matches `$NO_PROXY` semantics; `*` means bypass all).
`--proxy-insecure`	Skip cert verification on the TLS-to-proxy connection.
`--proxy-cacert <PATH>`	Extra CA for the proxy connection.
`--proxy-capath <DIR>`	Directory of `.pem`/`.crt`/`.cer` CAs for proxy TLS. Mirrors `--capath`.

Flag	Description
`--proxy-ca-native`	Disable webpki roots for proxy TLS; use OS native roots. Mirrors `--ca-native`.
`--proxy-pass <PASS>`	Passphrase for `--proxy-key` (HTTPS proxy mTLS). Accepted for curl parity; **deferred** — reqwest 0.12 does not expose a passphrase API for proxy mTLS. Emits a runtime warning.

Examples

```
recon -x http://proxy.corp:3128 https://api.example.com/
recon -x socks5h://127.0.0.1:9050 https://example.onion/
recon -x https://proxy.example.com:8443 -U alice:s3cr3t
https://example.com/
HTTPS_PROXY=http://proxy.corp:3128 NO_PROXY='.internal' recon
https://api.example.com/

# Proxy CA configuration (0.72.0)
recon --proxy http://corp-proxy:3128 --proxy-capath /etc/pki/proxy/
https://example.com/
recon --proxy https://proxy:8443 --proxy-ca-native https://example.com/

# Script equivalent
recon --script - <<'EOF'
http("https://api.example.com/", #{
    proxy: "socks5h://127.0.0.1:9050",
    proxy_user: "alice:s3cr3t",
    noproxy: ".internal",
});
EOF
```

Unix-domain sockets

0.51.0. Route the HTTP request over a Unix socket instead of TCP. Host header and path are preserved; only the transport changes.

Flag	Description
`--unix-socket <PATH>`	Socket path.

Examples

```
# Docker API
recon --unix-socket /var/run/docker.sock http://localhost/_ping
recon --unix-socket /var/run/docker.sock -p http://localhost/v1.40/version
recon --unix-socket /var/run/docker.sock
http://localhost/v1.40/containers/json

# systemd-activated service
recon --unix-socket /run/app.sock http://localhost/api/v1/status
```

HSTS persistent cache

0.52.0. A curl-compatible HSTS cache that upgrades `http://` to `https://` before sending (when the host has a non-expired cache entry) and updates itself from `Strict-Transport-Security` response headers.

Flag	Description
`--hsts <FILE>`	Load + update the HSTS cache at this path.

Examples

```
# Populate from an https:// response
recon --hsts ~/.recon/hsts.txt https://www.cloudflare.com/

# Future http:// requests to the same host are auto-upgraded
recon --hsts ~/.recon/hsts.txt http://www.cloudflare.com/
# * HSTS: upgrading http:// to https:// for www.cloudflare.com

# File format matches curl's --hsts:
cat ~/.recon/hsts.txt
# # HSTS cache (recon 0.58.1)
# # host expires_unix   (leading . = includeSubDomains)
# .www.cloudflare.com 1808493843
```

IPFS / IPNS

0.49.0. `ipfs://<cid>` and `ipns://<name>` URLs are rewritten to an HTTP gateway URL before the request fires.

Flag	Description
`--ipfs-gateway <URL>`	Gateway base URL. Default `https://ipfs.io`. Also read from `$RECON_IPFS_GATEWAY`.

Examples

```
recon ipfs://QmYwAPJzv5CZsnA625s3Xf2nemtYgPpHdWEz79ojWnPbdG
recon --ipfs-gateway http://127.0.0.1:8080 ipfs://bafy...   # local Kubo
node
recon ipns://example.eth                                     # via the
default gateway
```

Cookies

Flag	Description
`-b, --cookiejar <PATH>`	SQLite cookie jar at PATH (created on demand).
`--cookies`	Read-only listing of the current cookie jar.
`--cookie-set <NAME=VAL>`	Set a cookie in the jar.
`--cookie-delete <NAME>`	Remove by name.

Cookies persist across invocations when you pass the same `--cookiejar` path. The `.db` format is SQLite; inspect with `sqlite3` or any sqlite viewer.

Examples

```
# Login, save cookies, browse with them
recon https://example.com/login -d 'user=alice&pass=secret' --cookiejar
~/jar.db
recon https://example.com/dashboard --cookiejar ~/jar.db -p

# Inspect
recon --cookiejar ~/jar.db --cookies

# Inject manually
recon --cookiejar ~/jar.db --cookie-set 'session=abc123;
Domain=.example.com; Path=/'
```

DNS

Flag	Description
`--dns`	Resolve A, AAAA, MX, TXT, NS, CNAME, SOA for the URL's host.
`--dns-type <LIST>`	Comma-separated subset: `A,AAAA,MX,TXT,NS,CNAME,SOA,PTR,CAA,DS,DNSKEY,HTTPS,SRV,SVCB`.

Flag	Description
`--dns-servers <LIST>`	Comma-separated resolver IPs.
`--dns-ipv4-addr <IP>`	Bind DNS queries to a specific local IPv4.
`--dns-ipv6-addr <IP>`	Same for IPv6.
`--dns-interface <NAME>`	Accepted at the CLI; not yet plumbed (see OUT-OF-SCOPE.md). Use `--dns-ipv4-addr` / `--dns-ipv6-addr` as a workaround.

Examples

```
recon example.com --dns
recon example.com --dns-type A,AAAA,MX
recon example.com --dns --dns-servers 1.1.1.1,8.8.8.8
recon example.com --dns --dns-servers 127.0.0.1:5353        # local resolver on custom port
```

TLS certificate inspection

Standalone mode selected by `--cert` (without a `--cert <PATH>` value — it's the bool form of the flag).

Flag	Description
`--cert`	Inspect the server cert: subject, issuer, SANs, NotBefore/After, fingerprints.
`--cert-chain`	Walk the full intermediate chain.
`--sni <HOSTNAME>`	Override the SNI value sent in the handshake.

Examples

```
recon https://example.com --cert
recon https://example.com --cert --cert-chain
recon https://example.com --cert --sni alt.example.com        # request cert for alt via SNI
```

Ping, traceroute, netstatus

Stand-alone modes.

Flag	Description
`--ping <HOST>`	TCP ping by default, ICMP with `--icmp`.
`--icmp`	Force ICMP (needs raw sockets; root on Linux, CAP_NET_RAW elsewhere).
`--ping-count <N>`	Packets (default 4).
`--traceroute <HOST>`	Same as `--ping HOST --traceroute`.
`--max-hops <N>`	Hop limit (default 30).
`--netstatus`	Run the probe bundle defined in `~/.recon/config.toml [netstatus]`.

Examples

```
recon --ping 8.8.8.8
recon --ping example.com --ping-count 10
recon --ping example.com:443 --icmp        # ICMP with explicit port hint
recon --traceroute 8.8.8.8 --max-hops 20
recon --netstatus                           # connectivity sweep
```

Email protection

A single URL can be probed for its whole email-auth stack in one invocation.

Flag	Description
`--spf`	SPF record validation (recursive include/redirect, lookup limit)
`--dmarc`	DMARC policy + record
`--dkim <SELECTOR>`	DKIM for the given selector. Repeatable.
`--mta-sts`	MTA-STS policy

Flag	Description
`--bimi [SELECTOR]`	BIMI record + optional VMC / SVG fetch
`--tls-rpt`	SMTP TLS-RPT record

Examples

```
# Full sweep
recon example.com --spf --dmarc --dkim selector1 --mta-sts --bimi --tls-rpt

# Single checks
recon example.com --spf
recon example.com --dkim selector1 --dkim selector2
recon example.com --bimi default              # explicit selector
```

SMTP

Stand-alone SMTP client. Send mail or just probe capabilities.

Flag	Description
`--mail-from` <ADDR>	Envelope sender. Required for send.
`--mail-to` <ADDR>	Envelope recipient. Repeatable.
`--mail-subject` <STR>	Subject header. Default "recon SMTP test".
`--mail-body` <STR>	Body. Accepts `@file`, `@-`.
`--mail-header` <H: V>	Extra header. Repeatable.
`--smtp-auth` <USER:PASS>	AUTH PLAIN → LOGIN chain.
`--smtp-helo` <NAME>	HELO/EHLO name. Default `recon.local`.
`--no-starttls`	Don't negotiate STARTTLS.
`--dkim-key` <PATH>	DKIM-sign outbound with this PEM key.
`--dkim-selector` <SEL>	DKIM selector.

Flag	Description
`--dkim-domain <DOM>`	DKIM domain (defaults to `--mail-from` domain).
`--mail-auth <ADDR>`	Append `AUTH=<ADDR>` to MAIL FROM (RFC 4954). Accepted but currently emits a warning — lettre 0.11 limitation, see OUT-OF-SCOPE.md.

Examples

```
# Probe only
recon smtp://mail.example.com:25

# Send
recon smtp://smtp.example.com:587 --mail-from me@example.com --mail-to you@example.com \
      --mail-subject 'Hi' --mail-body 'Test' --smtp-auth me:s3cr3t

# With DKIM signing
recon smtp://smtp.example.com:587 --mail-from me@example.com --mail-to you@example.com \
      --dkim-key ~/.dkim/default.pem --dkim-selector default
```

Mail retrieval (POP3, IMAP)

Flag	Description
`--stls`	POP3: upgrade via STLS after CAPA.
`--imap-peek`	IMAP: use BODY.PEEK (don't flip \Seen).

URLs: `pop3://`, `pop3s://`, `imap://`, `imaps://`.

Examples

```
recon pop3s://alice:s3cr3t@pop.example.com/
recon pop3://alice:s3cr3t@pop.example.com/ --stls
recon imaps://alice:s3cr3t@imap.example.com/INBOX
recon imaps://alice:s3cr3t@imap.example.com/INBOX/3        # fetch message 3
recon imaps://alice:s3cr3t@imap.example.com/INBOX --imap-peek
```

File transfer

0.47.0 shipped FTP/FTPS, SFTP, TFTP, Gopher. 0.71.0 plumbed all previously-stubbed flags through to their underlying protocol modules.

Protocol	URL scheme	Notes
FTP / FTPS	`ftp://`, `ftps://`	Anonymous by default; userinfo in URL or `-u`.
SFTP	`sftp://`	SSH-backed. `--privkey` for a specific key.
TFTP	`tftp://`	RFC 1350. `--tftp-blksize` for RFC 2348 block-size negotiation.
Gopher	`gopher://`, `gophers://`	Selector fetch.

FTP flags

Flag	Description
`--list-only`	Use `NLST` instead of `LIST` (filenames only).
`-Q, --quote <CMD>`	Run an FTP command before listing (repeatable).
`--ftp-skip-pasv-ip`	Use control-channel IP for PASV data, ignoring server-advertised PASV IP.
`--ftp-pasv` / `--disable-epsv` / `--disable-eprt`	Confirm passive mode (suppaftp 6 default).

TFTP flags

Flag	Description
`--tftp-no-options`	Confirm vanilla RFC 1350 mode (no RFC 2347 options).

Examples

```
# FTP — listing vs fetch
recon ftp://ftp.example.com/                                       #
directory listing
recon ftp://ftp.example.com/incoming/file.bin -o local.bin         #
retrieve
recon ftp://alice:s3cr3t@ftp.example.com/                          #
authenticated
recon ftp://ftp.example.com/ --list-only                           # NLST
(filenames only)
recon ftp://ftp.example.com/ -Q 'SITE CHMOD 755 pub'              # pre-
transfer command
recon ftp://ftp.example.com/ --ftp-skip-pasv-ip                    # NAT-
friendly PASV

# SFTP
recon sftp://me@example.com/home/me/data.csv -o data.csv
recon sftp://me@example.com:2222/srv/ --privkey ~/.ssh/ci_rsa

# TFTP
recon tftp://tftp.example.com/pxelinux.cfg/default
recon tftp://tftp.example.com/boot.img -o boot.img --tftp-blksize 8192
recon tftp://tftp.example.com/config --tftp-no-options             # RFC
1350 vanilla mode

# Gopher
recon gopher://gopher.floodgap.com/
recon gopher://gopher.floodgap.com/0/gopher/proxy
```

Other protocols

SSH, SCP

Flag	Description
`--pubkey <PATH>`	Path to SSH public key file (alias for `--ssh-pubkey`).

```
recon ssh://me@example.com -- uname -a                             # SSH
exec; prints stdout
recon scp://me@example.com/etc/motd -o motd                        # SCP
fetch
recon ssh://me@example.com --pubkey ~/.ssh/id_ed25519.pub -- whoami  #
explicit public key
```

Telnet

```
recon telnet://router.example.com:23
```

LDAP

```
recon ldap://ldap.example.com -H 'Bind: cn=admin,dc=example,dc=com'
recon ldaps://ldap.example.com                 # implicit-TLS
```

WebSocket

```
recon wss://echo.websocket.events                # handshake + Ping/Pong
```

RTSP

```
recon rtsp://camera.example.com/stream1          # OPTIONS
```

Dict (RFC 2229)

```
recon dict://dict.org/d:recon
```

NTP, Memcached, Redis, MQTT

```
recon ntp://pool.ntp.org                         # clock offset + rtt
recon memcache://cache.example.com/?stats        # text-protocol stats
recon redis://cache.example.com/                 # PING
recon redis://cache.example.com/ -X INFO         # arbitrary RESP command
recon mqtt://broker.example.com --mqtt-topic status --mqtt-message 'hello'
recon mqtts://broker.example.com --mqtt-user alice --mqtt-pass s3cr3t ...
```

Source comparison

0.53.0 shipped `--compare`. Diff two sources (URL / path / `-` stdin). HTTP(S) sources flow through the full request pipeline so every request flag (-H, -u, -L, -k, cookies, proxy, HSTS) applies.

Flag	Description
`--compare <A> <B>`	Two sources.
`--compare-format <FMT>`	`unified` (default), `summary`, `sxs`.
`--compare-context <N>`	Unified context lines (default 3).

Exit codes follow GNU `diff`: 0 identical, 1 differ, 2+ source-load error.

Examples

```
recon --compare one.json two.json
recon --compare https://api.example.com/v1/status ./baseline.json
curl -s https://a/ | recon --compare - ./baseline.json
recon --compare a.txt b.txt --compare-format summary
recon --compare a.txt b.txt --compare-format sxs
recon --compare a.json b.json --compare-context 5
```

Document conversions

0.58.0 shipped markdown / HTML / PDF. See also `--help docs`.

Flag	Description
`--md-to-html <SRC>`	Markdown → HTML via comrak. Output `-o PATH` or stdout.
`--md-to-pdf <SRC>`	Markdown → PDF (via agent-browser). `-o PATH` required.
`--html-to-pdf <SRC>`	HTML → PDF (via agent-browser).
`--toc`	Inject a linkable TOC.
`--toc-depth <N>`	Include headings up to H`N` (default 3).
`--toc-title <STR>`	TOC heading (default "Contents").
`--doc-title <STR>`	`<title>` + PDF metadata title.
`--doc-author <STR>`	Author field in PDF document properties.
`--doc-subject <STR>`	Subject field in PDF document properties.
`--doc-keywords <STR>`	Keywords field in PDF document properties (comma-separated).

Flag	Description
`--doc-css <PATH>`	Inline a custom stylesheet.
`--no-default-css`	Skip bundled default CSS.
`--gfm`	Enable GitHub-flavored extensions (tables, task lists, strikethrough, autolinks, footnotes, tagfilter).
`--unsafe-html`	Allow raw HTML passthrough (comrak's `unsafe_` flag). Needed for cover pages and explicit `<div class="page-break">` markers. Off by default; assume the markdown is trusted when on.
`--page-break-on-h1`	Start a new PDF page before every top-level `#` heading except the first. No visible effect in HTML output (Chrome's printToPDF honours the `break-before: page` rule).

Examples

```
recon --md-to-html README.md --toc --gfm -o README.html
recon --md-to-pdf CHANGELOG.md --toc --gfm --doc-title 'recon release notes' -o changelog.pdf
recon --html-to-pdf report.html -o report.pdf
curl -s https://example.com/doc.md | recon --md-to-html - --toc -o doc.html
recon --md-to-pdf notes.md --toc --doc-css print.css -o notes.pdf
recon --md-to-pdf notes.md --no-default-css --doc-css corp.css -o notes.pdf

# Book-style: cover page + chapter breaks
recon --md-to-pdf book.md --toc --gfm --unsafe-html --page-break-on-h1 \
      --doc-title 'My Book' -o book.pdf

# Full PDF metadata — verifiable via pdfinfo
recon --md-to-pdf report.md \
      --doc-title 'Q1 Results' \
      --doc-author 'Alice Smith' \
      --doc-subject 'Quarterly financial report' \
      --doc-keywords 'finance, Q1, 2026' \
      -o report.pdf
# pdfinfo report.pdf | grep -E '(Title|Author|Subject|Keywords)'
```

Cover pages

With `--unsafe-html`, raw HTML passes through the markdown parser verbatim. The bundled CSS styles `<div class="cover">` as a full-page centered block with an automatic page break after:

```
<div class="cover">

# My Document

<div class="subtitle">An illustrated guide</div>

<hr>

<div class="version">Version 1.0</div>
<div class="date">2026-04-24</div>
<div class="author">Alice Example</div>

</div>

# First chapter

Body text...
```

The cover block uses a `min-height: 90vh` flex container with centered content. `.subtitle`, `.version`, `.date`, `.author`, and `.meta` child classes pick up smaller, muted styling. A horizontal rule becomes a narrow divider.

Page breaks

Two routes:

1. **`--page-break-on-h1`** — every top-level `#` heading starts a new PDF page (except the first, which opens the document). No markdown changes required.

2. **Explicit markers (needs `--unsafe-html`)** — embed one of:

   ```
   <div class="page-break"></div>
   <hr class="page-break">
   ```

   anywhere in the markdown. The CSS `break-after: page` kicks in.

Chrome's printToPDF is the renderer for both `--md-to-pdf` and `--html-to-pdf`, so any CSS that speaks `break-before`, `break-after`, or `page-break-*` works. `@page` rules for size and margins (defined in the bundled default CSS) are honored too — override via `--doc-css` or inline `<style>` with `--unsafe-html`.

PDF page export

Flag	Value	Description
`--export-pdf-page`	`<PAGE> <PDF>`	Render the 1-indexed PAGE of PDF as a raster image. Requires `pdftoppm` (poppler-utils).
`--pdf-viewport`	`<WxH>`	Target image box in pixels. Default `1024x1366`. Aspect is preserved — this is an upper-bound box.
`--pdf-scale`	`<N>`	Density multiplier (≥ 1). Default `2`. Final image fits within `W*N × H*N` px.
`--pdf-quality`	`<0-100>`	JPEG/WEBP quality. Default `90`.
`--pdf-format`	`<png|jpeg|webp>`	Override output format inference.

Examples:

```
recon --export-pdf-page 1 docs/MANUAL.pdf
# → page-1.png in CWD

recon --export-pdf-page 3 report.pdf -o cover.jpg --pdf-quality 75
recon --export-pdf-page 1 doc.pdf -o cover.webp --pdf-viewport 1920x2715
```

Barcode encoding & decoding

Flag	Description
`--encode <FORMAT>`	qr, datamatrix, code128, code39, ean13, upca, aztec, pdf417.
`--encode-format <FMT>`	ascii (default), svg, png. Inferred from `-o` extension.
`--from-file <PATH>`	Encode input from file (mutually exclusive with positional).
`--encode-list`	List all supported formats.
`--qr-level <L|M|Q|H>`	QR error-correction level (default M).
`--encode-hints <KEY=VAL>`	rxing encoder hint (repeatable). Applies to aztec / pdf417. Keys: `charset`, `eclevel`, `aztec-layers`, `pdf417-compact`, `pdf417-compaction`, `pdf417-auto-eci`, `margin`.
`--hrt` / `--no-hrt`	Human-readable text under 1D barcodes. Default on for EAN/UPC, off for Code128/39. SVG + ASCII only (PNG HRT deferred).

Flag	Description
`--decode <IMAGE>`	Scan a PNG/JPEG/WebP for any supported format. `-` for stdin.
`--decode-hints <LIST>`	Comma-separated format restriction.
`--decode-all <IMAGE>`	Scan for every barcode in the image; one line per detection.

Examples

```
# Encode
recon --encode qr -o qr.png 'https://example.com'
recon --encode qr --qr-level H -o durable.png 'sticker text'
recon --encode pdf417 -o id.png 'stacked linear code'
recon --encode aztec -o ticket.png 'transit ticket'
recon --encode ean13 --encode-format svg '4006381333931' > product.svg
recon --encode qr --from-file msg.txt -o qr-of-file.png

# Decode
recon --decode ticket.png                           # → aztec<TAB>transit ticket
cat code.png | recon --decode -
recon --decode mystery.png --decode-hints qr,datamatrix
recon --decode bottle.jpg --decode-hints ean13
recon --decode-all sheet.png                        # every code in the image
```

`--encode-hints` — Aztec / PDF417 tuning

`--encode-hints KEY=VAL` exposes the rxing `encode_with_hints` API for the two formats recon routes through rxing (Aztec and PDF417). The flag is repeatable; unknown keys error, as do hints applied to non-rxing formats (qr, datamatrix, code128, code39, ean13, upca).

Supported keys:

Key	Applies to	Value
`charset`	aztec, pdf417	Character set name driving the ECI selection (e.g. `UTF-8`, `Shift_JIS`, `ISO-8859-1`).
`eclevel`	aztec, pdf417	Aztec: minimum % of EC words. PDF417: integer `0..8` (higher = more redundancy).
`aztec-layers`	aztec	`-4..-1` for compact mode, `0` for auto, `1..32` for full mode.

Key	Applies to	Value
`pdf417-compact`	pdf417	`true` / `false`. Use PDF417 compact mode.
`pdf417-compaction`	pdf417	Compaction mode name (e.g. `TEXT`, `BYTE`, `NUMERIC`).
`pdf417-auto-eci`	pdf417	`true` / `false`. Auto-insert ECIs for non-Latin-1 input.
`margin`	aztec, pdf417	Quiet-zone margin in pixels.

```
# Compact Aztec (negative layer count)
recon --encode aztec --encode-hints aztec-layers=-2 'compact transit ticket'

# PDF417 with maximum EC redundancy and explicit UTF-8 ECI
recon --encode pdf417 \
      --encode-hints eclevel=8 \
      --encode-hints charset=UTF-8 \
      -o id.svg 'license payload'

# Aztec with Shift_JIS ECI for a Japanese payload
recon --encode aztec --encode-hints charset=Shift_JIS '    '
```

HRT (human-readable text under 1D barcodes)

EAN-13 and UPC-A get HRT by default; Code128 and Code39 don't unless you pass `--hrt`. Implemented for ASCII and SVG output. PNG output ignores the flag — PNG HRT is deferred pending bundled-font work.

```
recon --encode ean13 --encode-format svg '4006381333931' -o retail.svg
recon --encode code128 --hrt --encode-format svg 'SHIP-4711' -o box.svg
recon --encode ean13 --no-hrt '4006381333931' -o bare.svg
```

Hashing

Flag	Description
`--hash <ALGO>`	md5, sha1, sha224, sha256, sha384, sha512, sha3_256, sha3_512, blake3.
`--hash-list`	List supported algorithms.

Examples

```
recon --hash sha256 /etc/hosts
recon --hash sha256 https://example.com/file.iso          # hash a URL
cat payload.bin | recon --hash blake3 -
echo -n "hello" | recon --hash md5 -
```

Compression & archiving

Flag	Description
`--compress <ALGO>`	gzip, deflate, brotli, zstd, bzip2, lz4, xz, snap.
`--decompress <ALGO>`	Same set; auto-detect via magic bytes if ALGO is `auto`.
`--compress-list`	List supported algorithms.
`--compression-level` <N>	1..22 (depends on algo).
`--archive <DEST>`	Create zip/tar/tar.gz/tar.xz/tar.bz2. Positional args after DEST are sources.
`--extract <SRC>`	Extract an archive. `-o DIR` to choose destination.

Examples

```
recon --compress gzip /etc/hosts -o hosts.gz
recon --decompress auto hosts.gz -o hosts.plain
recon --compress zstd --compression-level 19 big.bin -o big.zst

recon --archive backup.tar.gz ~/Documents ~/src
recon --extract backup.tar.gz -o restore/
recon --extract release.zip
```

Encryption

age-based + PGP shell-out.

Flag	Description
`--encrypt`	Encrypt stdin / file to age format. Requires `--recipient` or `--pgp-recipient`.
`--decrypt`	Decrypt age / PGP. Needs `--identity` or a configured GPG keyring.

Flag	Description
`--encrypt-keygen`	Generate a new age keypair.
`--recipient <KEY>`	Age recipient (X25519 or ssh-ed25519). Repeatable.
`--identity <PATH>`	Age identity file for decryption.
`--passphrase`	Use passphrase-based (scrypt) encryption. Mutually exclusive with `--recipient`.
`--armor`	Armored (base64 PEM) output.
`--pgp-recipient <EMAIL|FPR>`	PGP recipient (shells out to `gpg`).

Examples

```
# Generate
recon --encrypt-keygen > age.key        # public key printed to stderr

# Encrypt
recon --encrypt --recipient age1abc... README.md -o README.md.age
recon --encrypt --passphrase secret.txt -o secret.txt.age --armor

# Decrypt
recon --decrypt --identity age.key README.md.age -o README.md

# PGP
recon --encrypt --pgp-recipient alice@example.com msg.txt -o msg.txt.pgp
```

Check digits

Flag	Description
`--checkdigit <ALGO>`	Verify an input.
`--checkdigit-create <ALGO>`	Compute and append the check digit.
`--checkdigit-list`	List supported algorithms (70+).

Notable algorithms shipped in 0.61.0:

- **Latin-American + Australian + Mexican tax IDs**: `br_cpf`, `br_cnpj`, `ar_cuit`, `ar_cuil`, `cl_rut`, `pe_ruc`, `au_abn`, `mx_rfc`. All except ABN support `--checkdigit-create`; ABN's mod-89 algorithm has no single-digit inverse.
- **110+ year warnings** on Nordic + Bulgarian personal IDs: `cpr` (Denmark), `henkilotunnus` (Finland), `fodselsnummer` (Norway), `bg-egn` (Bulgaria). Same idiom

as the Swedish `personnummer`: when the parsed birth year implies age ≥ 110, the verdict's `comment` flags a likely data-entry error.

Examples

```
recon --checkdigit isbn13 '9780131103627'          # → valid
recon --checkdigit-create isbn13 '978013110362'     # → 9780131103627
recon --checkdigit-create ean13 '400638133393'      # → 4006381333931
recon --checkdigit personnummer '19900101-1239'     # Swedish personal ID
recon --checkdigit-list | head -20
```

Sample data

Flag	Description
`--sample <NAME>`	Emit a sample payload (faker-style).
`--sample-list`	List all named samples.

Examples

```
recon --sample-list
recon --sample json-user
recon --sample json-user -o user.json
recon --sample css                                  # minimal CSS
boilerplate
```

JWT tokens

Flag	Description
`--jwt-view <TOKEN>`	Decode + pretty-print. No signature check.
`--jwt-sign <PAYLOAD>`	Sign a JWT. Pair with `--jwt-secret` or `--jwt-key`.
`--jwt-validate <TOKEN>`	Validate signature + standard claims.
`--jwt-secret <STR>`	HMAC secret (HS256/384/512).
`--jwt-key <PATH>`	RSA/ECDSA/Ed25519 PEM key.
`--jwt-alg <ALG>`	HS256, HS384, HS512, RS256, RS384, RS512, ES256, ES384, EdDSA.

Examples

```
recon --jwt-view "$(cat token.txt)"
recon --jwt-sign '{"sub":"alice","exp":9999999999}' --jwt-secret sharedkey
--jwt-alg HS256
recon --jwt-validate "$TOKEN" --jwt-secret sharedkey
recon --jwt-sign '{"sub":"alice"}' --jwt-key private.pem --jwt-alg RS256
```

Text encoding

Flag	Description
`--iconv <SRC:DST>`	Standalone `iconv` replacement.
`--list-charsets`	List supported charset labels.

Examples

```
recon --iconv utf-8:iso-8859-1 greek.txt -o greek-latin1.txt
echo 'Hello' | recon --iconv utf-8:utf-16le - -o hello.utf16
recon --list-charsets | head -20
```

Serve mode

Flag	Description
`--serve [ADDR]`	Start an HTTP server serving the cwd (default `127.0.0.1:8000`).
`--serve-tls [ADDR]`	HTTPS. Requires `--serve-cert` + `--serve-key`.
`--serve-cert <PATH>`	Server cert (PEM).
`--serve-key <PATH>`	Server key (PEM).
`--serve-sni <HOST:CERT:KEY>`	Per-hostname SNI mapping. Repeatable.

Examples

```
recon --serve
recon --serve 0.0.0.0:3000
recon --serve-tls 0.0.0.0:8443 --serve-cert cert.pem --serve-key key.pem
recon --serve-tls :443 --serve-sni a.example.com:a.pem:a.key --serve-sni
b.example.com:b.pem:b.key
```

Write-out format

The `-w '<FORMAT>'` flag prints a curl-compatible summary after the response. Variable names match curl's.

Variable	Description
`%{http_code}`	HTTP status code
`%{size_download}`	Body size in bytes
`%{size_header}`	Response header bytes
`%{size_request}`	Request bytes sent
`%{size_upload}`	Uploaded bytes
`%{speed_download}`	Bytes-per-second
`%{time_total}`	Total operation time (seconds, fractional)
`%{time_starttransfer}`	TTFB
`%{time_redirect}`	Time spent on redirects
`%{url}`	Final URL (after redirects)
`%{url_effective}`	Same as `%{url}`
`%{content_type}`	Response Content-Type
`%{num_redirects}`	Redirect count
`%{remote_ip}`	Final peer IP
`%{remote_port}`	Peer port
`%{local_ip}` / `%{local_port}`	Local side
`%{scheme}`	`http` or `https`

Not yet accurate: `time_namelookup`, `time_connect`, `time_appconnect`, `time_pretransfer` render as `0.000000` — see OUT-OF-SCOPE.md.

Examples

```
recon https://example.com/ -w '\n%{http_code} %{time_total}s %{size_download}B\n'
recon -I https://example.com/ -w '%{scheme}://%{remote_ip}:%{remote_port} %{http_code}\n'
recon -L https://short.url/x -w '%{num_redirects} hops → %{url_effective}\n'
```

Browser automation

Flag	Description
`--browser-screenshot <URL>`	Open in agent-browser, save a PNG to `-o PATH`.

Requires `agent-browser` on `$PATH`. See Script engine → agent-browser for deeper automation.

Examples

```
recon --browser-screenshot https://example.com -o example.png
```

Meta flags

Flag	Description
`--flags`	Alphabetical curl-style flag listing (`--help all` equivalent).
`--examples`	Curated examples, paged.
`--init`	Bootstrap `~/.recon/` (script/, jars/, sni/, config.toml).
`--editor`	Open response in `$EDITOR`.
`--editor-cleanup`	Remove leftover `~/.recon/editor-*` tempfiles.
`--no-pager`	Disable paging of `--help` / `--examples` / `--flags`.

`--flags` — the quick lookup

`recon --flags` prints every flag alphabetically sorted by long name, curl's `--help all` layout:

```
(short or 4-space pad) --long <VALUE>  short description
```

Short description is capped at ~52 characters (first sentence of each flag's internal help text). Use this as the quick index when you know roughly what you want but not the flag name; follow up with `recon --help <topic>` for the long-form deep-dive.

Example:

```
$ recon --flags | head
    --age                            Force the age backend, even if…
    --archive <DEST>                 Create an archive
    --armor                          Produce ASCII-armored output…
    --bimi <SELECTOR>                Validate the BIMI record
    --browser-screenshot <URL>       Open URL in a browser and save…
    --cacert <PATH>                  Path to a PEM-encoded CA certificate…
    --cert                           Fetch and display the server's TLS cert
    --cert-type <PEM|DER>            Format of --client-cert
-E, --client-cert <PATH>             Client certificate for mTLS
    --client-key <PATH>              Private key for the client certificate
```

The listing is auto-paged through `$PAGER`. Pipe to `grep` for search:

```
recon --flags | grep -i cookie
recon --flags | grep -E '^\s*-[a-zA-Z],'   # only flags with short keys
```

Interactive REPL

The `--repl` flag opens an interactive Rhai prompt backed by the script engine. Every binding available in `--script` mode is available at the prompt: `http()`, `hash_sha256()`, `encrypt_*`, `sqlite_*`, and so on.

Launching

Flag	Description
`--repl`	Open the REPL. Mutually exclusive with `--script` (REPL wins if both are given).
`--repl-history PATH`	Override the history file (default `~/.recon/repl_history`).

State persists across lines: `let` bindings and `fn` definitions remain in scope until you `:reset` or exit.

Meta-commands

All meta-commands start with `:` so they never collide with valid Rhai code.

Command	Behaviour
`:help`	Print the REPL cheat sheet.
`:help <topic>`	Print `recon --help <topic>` content without leaving the REPL.

Command	Behaviour
`:load <path>`	Eval `<path>` in the current scope. `let` / `fn` persist. Path resolves: literal then `~/.recon/script/<path>[.rhai]`.
`:run <path>`	Eval `<path>` in a fresh, throwaway scope. Prints the return value. REPL state untouched.
`:paste`	Enter paste mode. Lines accumulate until `:end` alone on a line. Then compile + eval once.
`:set <key> <val>`	Mutate flags. Keys: `method`, `header` (append), `timeout`, `user-agent`, `autoprint` (on/off).
`:vars`	List bound variables (consts and `let` bindings) with type tag and value preview.
`:fns`	List user-defined functions from the accumulated AST chain.
`:reset`	Clear user bindings and user functions. Keeps engine, history, and the const bindings (`args`, `flags`, …).
`:save <path>`	Write this session's successful input lines to `<path>` with a timestamp header.
`:save-tidy <path>`	Like `:save`, but appends missing `;` and drops entries that fail to compile so the saved file runs as a script.
`:functions [all]` / `:function-list`	List every callable registered with the engine (probes, helpers, builders). Pass `all` to also include the Rhai standard library.
`:history [N]`	Print the last `N` (default 20) inputs with 1-based indices.
`:!N`	Re-run history entry `N`.
`:edit`	Open `$EDITOR` (fallback `vi`) with a temp `.rhai` file. Eval the contents on save+quit.
`:time <expr>`	Evaluate `<expr>` and print the elapsed wall-clock time.
`:quit` / `:exit`	Save the history file and exit with code 0.

Multi-line input

The REPL detects unbalanced braces, parentheses, and strings and shows a `...` continuation prompt until the buffer parses. Use `:paste` to force a multi-line capture when the auto-detector mis-classifies pasted content; lines accumulate until `:end` on its own line.

Autoprint

Bare expressions print their result automatically (Python/Node convention). Toggle off with `:set autoprint off` if you want script-mode silence. `let` and `fn` declarations never print.

Threading caveat

`thread_spawn` is not available in REPL mode — it requires a static AST handle that the per-line REPL doesn't have. Calls return an error. Use `--script` for threaded workflows.

Example session

```
$ recon --repl
recon REPL — :help for commands, :quit to exit
>>> let response = http("https://api.example.com/users/1")
>>> response.status
200
>>> response.body.from_json().name
"Alice"
>>> fn pretty(r) { r.body.from_json().name }
>>> pretty(response)
"Alice"
>>> :vars
  let   response = #{status: 200, body: ...}
>>> :save investigation.rhai
saved 4 entries to investigation.rhai
>>> :quit
```

Part III — Script engine

recon ships an embedded Rhai interpreter. Every protocol probe, every cryptography primitive, every helper is exposed to scripts. Scripts run via `--script PATH`; `return N` from the top level becomes the process exit code.

Running scripts

```
recon --script ./probe.rhai                     # run an absolute / relative path
recon --script probe                             # falls back to ~/.recon/script/probe.rhai
recon --script probe https://example.com alice   # positional args → args[1], args[2]
```

Script lookup order:

1. Exact path (with or without `.rhai` extension).
2. `~/.recon/script/<name>.rhai`.
3. If neither exists — error.

The shipped example scripts in `script/*.rhai` are copy-friendly starting points:

```
cp script/*.rhai ~/.recon/script/
recon --script http example.com
```

Bare-word invocation

After `recon --init` + copying scripts to `~/.recon/script/`, scripts become first-class by name:

```
recon --script tcp-echo 127.0.0.1:9000
recon --script doc-convert README.md output.pdf
recon --script client-cert ~/keys/bundle.pem https://mtls.example.com/
```

Script language (Rhai)

Rhai is a lightweight embedded script language. Key facts for newcomers:

- **Let bindings**: `let x = 42;` (mutable by default; use `const` for compile-time constants).
- **Types**: integers (i64), floats (f64), bools, strings, arrays, maps (object literals: `#{ key: value }`), Blob (Vec).
- **String interpolation**: `` `Hello ${name}` ``.
- **Closures / function pointers**: `|a, b| { a + b }`.
- **For loops**: `for i in 0..5 { ... }` (integer ranges), `for item in array { ... }`, `for (k, v) in map { ... }`.
- **Functions**: `fn f(a) { a * 2 }`.
- **Imports**: `import "module-name" as m;` — resolved against the script's directory and `~/.recon/script/`.
- **Error handling**: `try { ... } catch(e) { ... }`.
- **Reserved words** that might surprise: `spawn` (use `thread_spawn`), `async`, `await`, `throw` (all reserved even when unused).

Useful idioms:

```
// Map with computed keys
let cfg = #{
    user: env("USER"),
    host: "api.example.com",
    timeout_ms: 5000,
};

// Chainable array methods
let evens = (0..20).filter(|n| n % 2 == 0).map(|n| n * 10);

// Early return
if !file_exists("/etc/passwd") { return 1; }
```

Types and literals

Rhai's type system is fixed (no user-defined types at runtime). Recon scripts use the following built-in types:

Type	Rust equivalent	Example literals
`i64`	64-bit signed integer	`42`, `-7`, `0xFF`, `0b1010`, `0o77`
`f64`	64-bit float	`3.14`, `-1.0e-5`, `1_000.5`
`bool`	Boolean	`true`, `false`
`()`	Unit / null	`()`
`char`	Unicode scalar	`'a'`, `'\n'`
`String`	UTF-8 string	`"hello"`, `` `hi ${name}` ``

Type	Rust equivalent	Example literals
Array	Heterogeneous dynamic array	`[1, "two", 3.0]`
Map	String-keyed object	`#{ x: 1, y: "ok" }`
Blob	Byte array (`Vec<u8>`)	`Blob()` , `"hello".to_blob()`

```
let n    = 255;               // i64 — integer literal
let f    = 3.14;              // f64
let flag = true;              // bool
let nil  = ();                // unit — Rhai's null
let ch   = 'A';              // char
let s    = "hello";           // string
let tmpl = `n is ${n}`;      // interpolated string (backtick)
let arr  = [1, "two", false]; // heterogeneous array
let obj  = #{ host: "localhost", port: 8080 };  // map
let raw  = "hello".to_blob(); // Blob of UTF-8 bytes
```

Integer literals accept `_` separators (`1_000_000`) and radix prefixes `0x` (hex), `0b` (binary), `0o` (octal). Type annotations (`let x: i64 = 0;`) are optional — Rhai infers the type.

Operators

Arithmetic

Operator	Description	Example
`+`	Addition; string concatenation	`1 + 2` , `"a" + "b"`
`-`	Subtraction	`10 - 3`
`*`	Multiplication	`4 * 5`
`/`	Division (integer truncates)	`7 / 2` → `3` ; `7.0 / 2.0` → `3.5`
`%`	Remainder	`10 % 3` → `1`
`**`	Power	`2 ** 10` → `1024`
`-x`	Unary negation	`-x`

Comparison (all return `bool`)

`==` , `!=` , `<` , `<=` , `>` , `>=` — work on all numeric types and strings (lexicographic for strings).

Logical

Operator	Description
`&&`	Short-circuit AND
`||`	Short-circuit OR
`!`	Logical NOT

Bitwise (integers only)

`&` (AND), `|` (OR), `^` (XOR), `~` (bitwise NOT), `<<` (left shift), `>>` (right shift).

String concatenation

`"hello " + "world"` — either operand may be a non-string and is automatically converted: `"n=" + 42` → `"n=42"`.

Null-coalescing

`value ?? default` evaluates to `default` when `value` is `()`, otherwise `value`. Useful for optional map lookups — accessing a missing key returns `()`:

```
let port    = opts["port"] ?? 80;
let charset = response["content-type"] ?? "application/octet-stream";
```

Note: `env()` always returns a `String` (empty string when unset), never `()`, so `??` does not apply to it. Use the two-argument form for env defaults: `env("HOME", "/tmp")`.

Ranges

`1..5` (exclusive — does not include 5), `1..=5` (inclusive — includes 5). Ranges are iterable and support `.filter()`, `.map()`, `.reduce()`.

Combined example

```
let x = 7;
let y = 2;
print(x + y);            // 9
print(x ** y);           // 49
print(x % y);            // 1
print(x > 5 && y < 10); // true
print((x | y) == 7);     // true (0b111 | 0b010 = 0b111)
let label = env("LABEL", "default");   // env() never returns () — use 2-
arg form
print(`label is ${label}`);
let sum = (1..=10).reduce(|acc, n| acc + n, 0);
print(sum);              // 55
```

Control flow

if / else if / else

```
if x > 10 {
    print("big");
} else if x > 0 {
    print("small");
} else {
    print("non-positive");
}
```

If blocks are expressions and return the last evaluated value:

```
let label = if x > 0 { "positive" } else { "non-positive" };
```

Ternary operator

```
let sign = x > 0 ? "+" : "-";
```

while

```
let i = 0;
while i < 5 {
    print(i);
    i += 1;
}
```

loop

`loop { ... }` runs until `break`:

```
let attempts = 0;
loop {
    attempts += 1;
    if attempts >= 3 { break; }
}
```

for — range

```
for i in 0..5  { print(i); }  // 0 1 2 3 4 (exclusive)
for i in 1..=5 { print(i); }  // 1 2 3 4 5 (inclusive)
```

for — array

```
let hosts = ["a.com", "b.com", "c.com"];
for host in hosts {
    print(host);
}
```

for — map

```
let cfg = #{ host: "a.com", port: 443 };
for (key, val) in cfg {
    print(`${key} = ${val}`);
}
```

break / continue

```
for i in 0..10 {
    if i == 3 { continue; }  // skip 3
    if i == 7 { break; }     // stop before 7
    print(i);
}
```

return

`return` exits the current function (or the script when used at top level):

```
fn first_positive(arr) {
    for n in arr {
        if n > 0 { return n; }
    }
    ()  // not found — implicit unit return
}
```

Functions and closures

Named functions

```
fn add(a, b) { a + b }        // last expression is implicit return value

fn factorial(n) {
    if n <= 1 { return 1; }
    n * factorial(n - 1)      // recursion works
}

print(add(3, 4));      // 7
print(factorial(5));   // 120
```

Rhai hoists function definitions, so a function can be called before it appears in the file.

Closures

```
let double = |x| x * 2;
let greet  = |name| `Hello, ${name}!`;

print(double(5));       // 10
print(greet("world"));  // Hello, world!

let nums   = [1, 2, 3, 4, 5];
let evens  = nums.filter(|n| n % 2 == 0);  // [2, 4]
let square = evens.map(|n| n * n);          // [4, 16]
print(square);
```

Closures capture outer variables by value at creation time.

Function pointers

```
fn triple(n) { n * 3 }

let fp = Fn("triple");    // function pointer by name
print(call(fp, 7));       // 21
```

Error handling

try / catch

In Rhai 1.x, `e` in a `catch` block is the error message as a string:

```
try {
    let r = http("https://httpbin.org/status/500");
    if r.status != 200 {
        throw `HTTP error: ${r.status}`;
    }
    print(r.body);
} catch(e) {
    eprint(`Error: ${e}`);
}
```

throw

`throw "message"` raises an error that propagates to the nearest `catch` block or terminates the script with a non-zero exit code:

```
fn parse_port(s) {
    let n = s.to_int();
    if n < 1 || n > 65535 { throw `invalid port: ${s}`; }
    n
}
```

assert

`assert(cond, msg)` is a recon helper that calls `throw msg` when `cond` is `false`. Prefer it over manual if/throw when validating invariants:

```
let r = http("https://httpbin.org/get");
assert(r.status == 200, `Expected 200, got ${r.status}`);
```

Standard built-in functions

The following are available in every recon script without any import. They come from Rhai's standard packages, augmented by recon's own helpers.

Type inspection and conversion

Expression	Description	Example result
`type_of(v)`	Runtime type name	`"i64"`, `"f64"`, `"string"`, `"array"`, `"map"`, `"bool"`, `"()"`, `"blob"`
`v.to_int()`	Convert to `i64`	`"42".to_int()` → `42`
`v.to_float()`	Convert to `f64`	`3.to_float()` → `3.0`
`v.to_string()`	Convert to `String`	`true.to_string()` → `"true"`
`parse_int(s)`	Parse decimal string → `i64`	`parse_int("255")` → `255`
`parse_float(s)`	Parse string → `f64`	`parse_float("3.14")` → `3.14`

```
let v = "123".to_int();
print(type_of(v));            // "i64"
print(type_of(3.14));         // "f64"
print(type_of([1, 2]));       // "array"
print(type_of(()));           // "()"
print(42.to_float() + 0.5);  // 42.5
print(parse_int("0xFF"));     // 255
```

String methods

Strings in Rhai are immutable. Methods that appear to "modify" a string return a new one.

Method	Description
`.len()`	Character (code-point) count
`.is_empty()`	`true` if `""`
`.to_upper()`	Uppercase copy
`.to_lower()`	Lowercase copy
`.trim()`	Strip leading and trailing whitespace
`.trim_start()` / `.trim_end()`	Strip one side only
`.contains(sub)`	Substring check; returns `bool`
`.starts_with(s)` / `.ends_with(s)`	Prefix / suffix check
`.replace(old, new)`	Replace all occurrences
`.split(sep)`	Split on separator; returns array
`.chars()`	Array of single-character strings
`.sub_string(start)`	Slice from index to end
`.sub_string(start, len)`	Slice of given length
`.pad(len, ch)`	Right-pad to `len` with char `ch`
`.index_of(sub)`	First match index or `-1`
`.to_blob()`	Encode as UTF-8 `Blob`
`.to_int()`	Parse as decimal integer
`.to_float()`	Parse as float

```
let raw = "  https://Example.COM/Path  ";
let url = raw.trim().to_lower();          // "https://example.com/path"
print(url.contains("example"));          // true
print(url.starts_with("https"));         // true
print(url.replace("example.com", "cdn.example.com"));

let parts = "a,b,c,d".split(",");        // ["a", "b", "c", "d"]
print(parts.len());                       // 4
print(parts.join(" | "));                // "a | b | c | d"

let host = "api.example.com";
print(host.sub_string(4, 7));            // "example"
print(host.index_of("."));              // 3
```

Array methods

Method	Description
`.len()`	Number of elements
`.is_empty()`	`true` if array is empty
`.push(v)`	Append element (in-place)
`.pop()`	Remove and return last element
`.shift()`	Remove and return first element
`.unshift(v)`	Prepend element (in-place)
`.insert(i, v)`	Insert at index `i` (in-place)
`.remove(i)`	Remove element at index `i` and return it
`.reverse()`	Reverse in-place
`.contains(v)`	Membership test
`.index_of(v)`	First index of `v`, or `-1`
`.join(sep)`	Concatenate elements as strings with separator
`.map(fn)`	Apply function; returns new array
`.filter(fn)`	Keep elements where predicate is `true`; returns new array
`.reduce(fn, init)`	Fold to single value
`.sort()`	In-place ascending sort
`.sort(fn)`	In-place sort with comparator `fn(a, b) -> i64`
`.dedup()`	Remove consecutive duplicate elements (in-place)
`.drain(range)`	Remove and return elements in index range

```
let nums = [5, 3, 8, 1, 9, 2];
nums.sort();                                   // [1, 2, 3, 5, 8, 9]

let big  = nums.filter(|n| n > 4);           // [5, 8, 9]
let dbl  = big.map(|n| n * 2);               // [10, 16, 18]
let sum  = dbl.reduce(|acc, n| acc + n, 0);  // 44

nums.push(42);
nums.unshift(0);
print(nums.len());           // 8
print(nums.contains(42));    // true
print(nums.index_of(3));     // 2 (after sort)

// Negative indices index from the end
print(nums[-1]);             // last element
```

Map methods

Maps use string keys. Literal syntax is `#{ key: value }`. Keys with special characters or spaces must be quoted: `#{ "content-type": "json" }`.

Method / accessor	Description
`.len()`	Number of key-value pairs
`.is_empty()`	`true` if no entries
`.keys()`	Array of all key strings
`.values()`	Array of all values
`.contains_key(k)`	Key existence check
`.remove(k)`	Delete key and return its value
`m.key` / `m["key"]`	Read a value
`m.key = v` / `m["key"] = v`	Write a value

```
let hdr = #{
    "content-type": "application/json",
    accept: "application/json",
};
print(hdr.len());                         // 2
print(hdr.keys());                        // ["content-type", "accept"]
print(hdr.contains_key("accept"));        // true

hdr["x-request-id"] = "abc-123";

for (k, v) in hdr {
    print(`${k}: ${v}`);
}

let ct = hdr.remove("content-type");
print(ct);                                // "application/json"
print(hdr.len());                         // 2
```

Math functions

Function	Description
`abs(n)`	Absolute value
`sqrt(n)`	Square root (returns `f64`)
`pow(base, exp)`	Power — equivalent to `base ** exp`
`min(a, b)` / `max(a, b)`	Minimum / maximum
`round(n)`	Round to nearest integer

Function	Description
`floor(n)` / `ceil(n)`	Floor / ceiling
`sin(n)` / `cos(n)` / `tan(n)`	Trig functions (radians)
`asin(n)` / `acos(n)` / `atan(n)`	Inverse trig (radians)
`ln(n)`	Natural logarithm (base e)
`log(n)`	Base-10 logarithm
`exp(n)`	e^n
`PI`	$\pi \approx 3.14159265358979$
`E`	Euler's number ≈ 2.71828182845905

```
print(abs(-7));            // 7
print(sqrt(144.0));        // 12.0
print(pow(2, 10));         // 1024
print(round(3.7));         // 4
print(floor(3.7));         // 3
print(ceil(3.1));          // 4
print(min(5, 3));          // 3
print(max(5, 3));          // 5

let angle = PI / 4.0;
print(sin(angle));         // ≈ 0.7071 (sin 45°)
print(exp(1.0));           // ≈ 2.7183 (same as E)
```

Ranges

Ranges are lazy integer sequences.

Expression	Meaning
`a..b`	Exclusive: integers `a`, `a+1`, …, `b-1`
`a..=b`	Inclusive: integers `a`, `a+1`, …, `b`

Ranges support `.filter()`, `.map()`, `.reduce()`, `.for_each()`. A range can also be used as an array slice index: `arr[start..end]`.

```
// Sum 1 to 100
let total = (1..=100).reduce(|acc, n| acc + n, 0);
print(total);   // 5050

// Collect even squares
let evens = (1..=10)
    .filter(|n| n % 2 == 0)
    .map(|n| n * n);
print(evens);   // [4, 16, 36, 64, 100]

// Array slice with range
let letters = ["a", "b", "c", "d", "e"];
let mid = letters[1..4];   // ["b", "c", "d"]
print(mid);
```

Shebang — executable scripts

A `.rhai` file can be made directly executable by adding a shebang as the first line:

```
#!/usr/bin/env -S recon --script
```

The `-S` flag instructs `/usr/bin/env` to split its argument on whitespace, so `recon` receives `--script` as a separate flag. This works on macOS and all major Linux distributions.

Full example

```
cat > ~/bin/health <<'EOF'
#!/usr/bin/env -S recon --script
let host = if args.len() > 1 { args[1] } else { "example.com" };
let r = https(`https://${host}`);
print(`${r.status} ${host} (${r.duration_ms}ms)`);
return if r.status == 200 { 0 } else { 1 };
EOF
chmod +x ~/bin/health

./health example.com          # run directly
./health api.example.com      # host from args[1]
```

How it works

When the kernel executes a shebang file, it calls the interpreter with the script path as an argument — equivalent to `recon --script ./health`. Recon detects a leading `#!` in the source and replaces it with `//` (a Rhai line comment) before compilation. This preserves line numbers in error messages while making the file valid Rhai.

Arguments and flags

Trailing arguments after the script name land in `args[1..]` exactly as with `--script`. CLI flags (`-k`, `-v`, `--connect-timeout`, etc.) set before or after the script name in the invocation are reflected in the `flags` map inside the script.

```
./health api.example.com         # args[1] = "api.example.com"
recon -k ./health api.example.com  # flags.insecure = true inside script
```

Notes

- On older systems where `/usr/bin/env -S` is not available, use the full path: `#!/usr/local/bin/recon --script` (hard-coded path, no `-S` needed).
- The shebang is only stripped when it appears on the **first line**. A `#!` appearing anywhere else in the file remains a parse error (Rhai does not treat `#` as a comment character).
- `recon --help shebang` summarises the shebang feature.

CLI inheritance

When a script runs, two read-only constants land at the top of its scope:

- **`args`** — an array of strings. `args[0]` is the script path; `args[1..]` are the positional arguments after `--script PATH`.
- **`flags`** — a map mirroring the relevant CLI flags (lower-case, snake_case keys). Useful for scripts that want to respect `-v`, `-k`, `--connect-timeout`, etc. at invocation time.

```
// Reasonable defaults, overridable via args
let url = if args.len() > 1 { args[1] } else { "https://example.com/" };
let timeout = flags.connect_timeout;   // from -C / --connect-timeout

let r = http(url, #{ timeout_ms: timeout * 1000 });
print(`${r.status} ${r.url}`);
```

Bindings that take their own opts map (e.g. `http(url, opts)`) layer the caller's opts ON TOP of the CLI defaults. If the caller wants to ignore the CLI's `-H` header list, they can pass `headers: []` in their opts.

Core helpers

Exposed by `src/script/bindings/helpers.rs`. Always available.

Function	Returns	Description
`sleep_ms(ms)`	—	Block the current thread.
`sleep(ms)`	—	Alias of sleep_ms (added in 0.56.0 for thread-side readability).
`env(name)`	string	Process environment variable; `""` if unset.
`env(name, default)`	string	With fallback.
`env_all()`	Map	Snapshot every process env var. Aliased as `envAll`.
`load_dotenv(path)`	int	Parse `.env` and set each KEY=VALUE in the process env (overrides existing). Returns count of vars set. Aliased as `loadDotEnv`.
`load_dotenv(path, override)`	int	Two-arg form: `false` leaves pre-existing env vars in place.
`script_path`	string (constant)	Resolved absolute path of the running script. Pushed into the Scope at startup.
`script_dir`	string (constant)	Parent directory of `script_path`. Use to load sibling files independent of CWD: `load_dotenv(script_dir + "/.env")`.
`script_name`	string (constant)	File stem of the running script (basename minus extension). Useful with the per-script overlay convention: `load_dotenv(script_dir + "/.env." + script_name)`.
`now()`	int	Unix seconds.
`now_ms()`	int	Unix milliseconds.
`assert(cond, msg)`	—	Throws on false.
`json_parse(s)`	Dynamic	Parse JSON text → native Rhai value.
`json_stringify(v)`	string	Serialize to compact JSON.
`json_stringify(v, pretty)`	string	`pretty: true` = 2-space indent.
`json_stringify(v, n)`	string	`n` = spaces per indent (clamped 1..=8).

Rhai's built-in `print(x)` writes `x` + newline via the engine's debug callback. recon adds byte-precise siblings (see next section).

Examples

```
// Probe a URL, fail fast
let r = http("https://api.example.com/health");
assert(r.status == 200, `expected 200, got ${r.status}`);

// JSON round-trip
let obj = json_parse(r.body);
print(json_stringify(obj, true));     // pretty

// Time gate
let t0 = now_ms();
let r = http("https://slow.example.com/");
let dt = now_ms() - t0;
print(`took ${dt} ms`);

// Polling with timeout
let deadline = now() + 30;
while now() < deadline {
    let r = http("https://example.com/ready");
    if r.status == 200 { return 0; }
    sleep_ms(500);
}
return 1;

// Layered .env loading: common defaults, per-script overrides.
// Use script_dir so the .env files don't depend on CWD — they live
// alongside the script in the same directory.
load_dotenv(script_dir + "/.env");                       // shared
load_dotenv(script_dir + "/.env." + script_name);        // per-script
overlay wins
print(env("LOG_LEVEL"));

// Non-override variant: shell-env wins over file values
load_dotenv(script_dir + "/.env", false);

// env_all() — useful for filtering or logging the entire env
let everything = env_all();
print(`process env has ${everything.len()} variables`);
```

`load_dotenv` overrides existing env values by default — that's what makes the `common.env, then .env.<scriptname>` pattern layer correctly. Pass `false` as a second arg to leave pre-existing values in place (e.g. when shell exports should take priority over file defaults). `std::env::set_var` is technically unsound under concurrent reads on some platforms, so call `load_dotenv` at the top of the script — before `thread_spawn` or any concurrent binding.

Output bindings

0.53.0. Byte-precise output that Rhai's built-in `print()` can't do.

Function	Description
`print_raw(s)` / `print_raw(blob)`	Write to stdout without a trailing newline; flush.
`eprint(s)`	Write to stderr with a newline.
`eprint_raw(s)` / `eprint_raw(blob)`	Write to stderr without newline; flush.
`flush()`	Explicit stdout flush.

Examples

```
// Progress indicator
print_raw("loading");
for i in 0..10 {
    sleep_ms(200);
    print_raw(".");
}
print_raw("\n");

// Send bytes to stdout (e.g. for piping into another tool)
let bytes = http("https://example.com/image.png").body;
print_raw(bytes);

// Log to stderr; keep stdout for data
eprint(`[${now_ms()}] starting probe…`);
print_raw(json_stringify(result));      // stdout stays parseable
```

File I/O bindings

0.53.0. Both whole-file convenience and a streaming handle API.

Whole-file helpers

Function	Description
`file_read(path)`	Read entire file → Blob.
`file_write_all(path, blob|str)`	Overwrite. Returns bytes written.
`file_append_all(path, blob|str)`	Append; creates if absent.
`file_exists(path)`	Bool.
`file_size(path)`	Bytes.
`file_delete(path)`	Unlink.

`path` accepts a filesystem path or a `file://` URL.

Streaming handle API

Function	Description
`file_open(path, mode)`	Returns a FileHandle. Modes: `r`, `w` (truncate+create), `rw`, `rwc` / `w+` (read+write+create+truncate), `a` (append+create), `ra` (append+read).
`file_read(h, n)`	Read up to `n` bytes → Blob.
`file_read_all(h)`	Drain to Blob.
`file_write(h, blob|str)`	Write all bytes. Returns count.
`file_seek(h, pos, whence)`	whence = `start` | `cur` | `end`. Returns new position.
`file_tell(h)`	Current position.
`file_flush(h)`	Sync buffered writes.
`file_close(h)`	Close (drops the underlying file).

Examples

```
// Whole-file
let data = file_read("config.json");
file_write_all("/tmp/copy.json", data);
file_append_all("/tmp/audit.log", `probe at ${now()}\n`);

// Check before read
if !file_exists("secrets.key") {
    eprint("missing key");
    return 2;
}

// Streaming — copy with 4KB chunks
let src = file_open("big.bin", "r");
let dst = file_open("/tmp/big.bin.copy", "w");
loop {
    let chunk = file_read(src, 4096);
    if chunk.len() == 0 { break; }
    file_write(dst, chunk);
}
file_close(src);
file_close(dst);

// Random access
let h = file_open("/tmp/log.bin", "rwc");
file_write(h, "record-1\n");
file_write(h, "record-2\n");
file_seek(h, 0, "start");
let head = file_read_all(h);
print(`contents: ${head.len()} bytes`);
file_close(h);
```

Clipboard bindings

0.70.0. The `clipboard` static module exposes the system clipboard for read/write access from Rhai scripts. Backed by the same `arboard` crate that powers the `--clipboard` family of CLI flags, so behaviour is identical across platforms (macOS pasteboard, Linux X11/Wayland, Windows).

Function	Returns	Description
`clipboard::get()`	string	Current clipboard text. Errors if no clipboard available or content isn't text.
`clipboard::set(text)`	()	Replace clipboard contents with `text`.

Examples

```
// Read, transform, write back to clipboard.
let original = clipboard::get();
let upper = original.to_upper();
clipboard::set(upper);
print("Replaced clipboard with uppercase form");
```

```
// Fetch a URL and place the prettified JSON response on the clipboard.
let r = http("https://api.example.com/data");
let pretty = json_stringify(json_parse(r.body.to_string()));
clipboard::set(pretty);
print("Result copied to clipboard");
```

Comparison bindings

0.53.0. Diff two in-memory values (strings or Blobs).

Function	Returns	Description
`compare(a, b)`	Map	`#{ identical, binary, a_bytes, b_bytes, added, removed, diff }`.

Both `a` and `b` accept strings OR Blobs. The binding does NOT fetch URLs — scripts should pre-load via `http()` or `file_read()` and pass the bytes.

Examples

```
// Compare two API responses
let a = http("https://api.v1.example.com/users/42").body;
let b = http("https://api.v2.example.com/users/42").body;
let r = compare(a, b);
if r.identical {
    print("parity OK");
    return 0;
}
print(`diverged: +${r.added} / -${r.removed}`);
print_raw(r.diff);
return 1;

// Compare a live URL against a baseline file
let live = http("https://example.com/status").body;
let baseline = file_read("status.baseline").to_string();
let r = compare(live.to_string(), baseline);
if !r.identical { print_raw(r.diff); }

// Binary content
let old = file_read("logo.v1.png");
let new = file_read("logo.v2.png");
let r = compare(old, new);
print(`binary: ${r.binary}, size delta: ${r.b_bytes - r.a_bytes}`);
```

HTTP binding

The workhorse. Two call forms:

```
http(url)           → response map
http(url, opts)     → response map
```

Response map

Key	Type	Description
`status`	int	HTTP status code.
`url`	string	Original URL.
`final_url`	string	After redirects.
`headers`	Map	Response headers; values are strings (first) or arrays when repeated.
`body`	Blob | string	Response body. String when content-type is text-ish; Blob otherwise.

Key	Type	Description
`duration_ms`	int	Total wall-clock time.
`http_version`	string	`HTTP/1.1`, `HTTP/2`, etc.

Options map

Every key is optional.

Key	Type	Description
`method`	string	GET, POST, PUT, PATCH, DELETE, HEAD.
`headers`	Map	Key → value (or array of values). Merged with CLI `-H` defaults.
`body`	string / Blob	Request body.
`json`	Dynamic	Serialize as JSON, set `Content-Type` + `Accept`.
`form`	Map	URL-encoded form body.
`multipart`	Array of Maps	Each Map has `name`, `value` or `file`, optional `filename`, `content_type`.
`basic_auth`	string	`"user:pass"`.
`bearer`	string	Bearer token.
`user_agent`	string	Override User-Agent.
`referer`	string	Referer.
`timeout_ms`	int	Total operation timeout.
`connect_timeout`	int	TCP connect timeout (seconds).
`insecure`	bool	Skip TLS verification.
`follow_redirects`	bool	Enable / disable redirect follow.
`max_redirects`	int	Redirect cap.
`compressed`	bool	Request + auto-decompress.
`cookiejar`	string	Path to a SQLite cookie jar.
`proxy`	string	Proxy URL.
`proxy_user`	string	Proxy basic auth.
`noproxy`	string	Bypass list.
`proxy_insecure`	bool	Skip cert verification on proxy.
`proxy_cacert`	string	Extra CA for proxy.

Key	Type	Description
`unix_socket`	string	Route via Unix socket.
`hsts`	string	Path to HSTS cache.
`client_cert`	string	PEM client cert.
`client_key`	string	PEM client key.
`cert_type` / `key_type`	string	PEM | DER | ENG (see CLI docs).
`pass`	string	PKCS#8 passphrase (reserved).
`wait`	int	(0.67.0) Seconds between URLs in batch mode (CLI-only effect).
`tries`	int	(0.67.0) Total attempts; overrides `retry`. `tries = retries + 1`.
`accept`	string	(0.67.0) Comma-separated filename-suffix accept list.
`reject`	string	(0.67.0) Comma-separated filename-suffix reject list.
`prettify`	bool	Pretty-print response body (auto-detect format).
`prettify_as`	string	Force prettify format (json/xml/html/yaml/csv/tsv/auto). Implies `prettify: true`.
`impersonate`	string	(0.77.0) Browser TLS+H2 fingerprint profile name (e.g. `"chrome_131"`, `"firefox_128"`, `"safari_17.5"`). Requires a build with `--features impersonate`; rejected with a rebuild hint otherwise. Hyphens accepted (`"chrome-131"` ≡ `"chrome_131"`). See `recon --help impersonate`.
`ja3`	string	(0.77.0, **deferred**) Reserved for raw JA3 ClientHello override. Errors at runtime as not-yet-implemented; use `impersonate` instead.
`ja4`	string	(0.77.0, **deferred**) Reserved for raw JA4 fingerprint override. Errors at runtime as not-yet-implemented.
`http2_fingerprint`	string	(0.77.0, **deferred**) Reserved for raw Akamai HTTP/2 fingerprint override. Errors at runtime as not-yet-implemented.

Examples

```
// Simple GET
let r = http("https://api.example.com/status");
print(`${r.status} ${r.body.len()} bytes`);

// POST JSON
let r = http("https://api.example.com/users", #{
    method: "POST",
    json: #{ name: "alice", email: "a@example.com" },
});
assert(r.status == 201, "create failed");

// Form POST
http("https://api.example.com/login", #{
    method: "POST",
    form: #{ user: "alice", pass: "s3cr3t" },
    cookiejar: "/tmp/jar.db",
});

// Multipart upload
http("https://upload.example.com/avatars", #{
    method: "POST",
    multipart: [
        #{ name: "title", value: "profile" },
        #{ name: "image", file: "avatar.png", content_type: "image/png" },
    ],
    bearer: env("API_TOKEN"),
});

// Bearer auth + timeout + compressed
let r = http("https://api.example.com/items", #{
    bearer: env("API_TOKEN"),
    timeout_ms: 5000,
    compressed: true,
    headers: #{ "Accept": "application/json" },
});

// Follow + retry on 5xx
for attempt in 0..3 {
    let r = http("https://api.example.com/", #{ follow_redirects: true });
    if r.status < 500 { return r.status == 200 ? 0 : 1; }
    sleep_ms(1000 * (attempt + 1));
}
return 2;

// Proxy + HSTS
http("http://example.com/", #{
    proxy: "socks5h://127.0.0.1:9050",
    hsts: "/tmp/hsts.txt",
```

```
});

// Browser fingerprint impersonation (0.77.0; requires --features
impersonate)
let r = http("https://tls.peet.ws/api/all", #{
    impersonate: "chrome_131",
});
print(`status: ${r.status}, body: ${r.body.len()} bytes`);

// mTLS
http("https://mtls.example.com/", #{
    client_cert: "/etc/keys/bundle.pem",
});

// Unix socket
let r = http("http://localhost/v1.40/version", #{
    unix_socket: "/var/run/docker.sock",
});
print(json_stringify(json_parse(r.body), 2));

// Pretty-print response body (force JSON format)
let r = http("https://api.example.com/data", #{ prettify_as: "json" });
print(r.body);
```

Browser (sticky-session) binding

A stateful HTTP "browser" — cookies and headers persist across calls against the same handle. Matches the common "log in once, then make authenticated requests" pattern.

Function	Description
`browser()`	New browser with a tempfile cookie jar.
`browser(opts)`	With initial config.
`use_persistent_session(name)`	Browser method; jar is `~/.recon/jars/<name>.db`.
` .get(url [, opts])`	GET via the browser.
` .post(url, opts)`	POST.
` .put(url, opts)` / `.patch(...)` / `.delete(url [, opts])`	Other methods.
` .request(method, url, opts)`	Catch-all.
` .header(name, value)`	Sticky request header.

Function	Description
` .headers(map)`	Merge sticky headers.
` .clear_headers()`	Remove sticky headers.
` .user_agent(str)`	Override User-Agent for the lifetime of the browser.
` .basic_auth(user, pass)`	Sticky Basic auth.
` .timeout_ms(n)` / `.connect_timeout(secs)`	Sticky timeouts.
` .insecure(bool)` / `.follow_redirects(bool)` / `.max_redirects(n)`	Sticky knobs.
` .cookiejar()`	Current jar path.
` .cookies()`	Array of cookie records from the jar.
` .cookie_set(name, value, domain, path)`	Inject a cookie.

Examples

```
// Login, then fetch protected resources
let br = browser();
br.user_agent("recon-test/0.58");
br.post("https://example.com/login", #{
    form: #{ user: "alice", pass: "s3cr3t" },
});
let r = br.get("https://example.com/dashboard");
assert(r.status == 200, "dashboard fetch failed");

// Persist across runs — next invocation sees the same cookies
let br = browser();
br.use_persistent_session("gh-session");
br.header("Accept", "application/vnd.github+json");
let r = br.get("https://api.github.com/user");

// Multi-persona fan-out
let personas = [#{ name: "a", jar: "persona-a" }, #{ name: "b", jar:
"persona-b" }];
for p in personas {
    let br = browser();
    br.use_persistent_session(p.jar);
    let r = br.get("https://api.example.com/whoami");
    print(`${p.name}: ${r.status}`);
}
```

TCP, TCP-server, UDP

0.57.0 shipped the server bindings. Handles are Send+Sync so they survive `thread_spawn` (0.56.0) boundaries — the expected pattern is "accept on main, spawn a handler per connection".

TCP client (existing)

Function	Description
`tcp(url)`	TCP connect probe; returns `#{ status, duration_ms, …}`.
`tcp(url, opts)`	With `timeout` (ms).

TCP server (0.57.0)

Function	Description
`tcp_listen(addr)`	Bind. `addr` like `"0.0.0.0:8080"` or `"[::]:8080"`.

Function	Description
`tcp_accept(listener)`	Blocking accept. Returns a TcpConn.
`tcp_accept(listener, timeout_ms)`	Times out with an error.
`tcp_read(conn, n, timeout_ms)`	Read up to N bytes → Blob.
`tcp_read_line(conn, timeout_ms)`	`\n`-terminated line; CR/LF stripped.
`tcp_write(conn, blob|str)`	Write all. Returns bytes.
`tcp_peer_addr(conn)`	Remote SocketAddr string.
`tcp_close(conn)`	Close connection.
`tcp_close_listener(l)`	Close listener.

UDP

Function	Description
`udp_bind(addr)`	Bind.
`udp_recv_from(sock, max_len)`	Block until a datagram arrives. Returns `#{ data: Blob, addr: string }`.
`udp_recv_from(sock, max_len, timeout_ms)`	With timeout.
`udp_send_to(sock, blob|str, addr)`	Returns bytes sent.
`udp_close(sock)`	Release.

Examples

```
// TCP client probe
let r = tcp("example.com:443");
print(`${r.status} in ${r.duration_ms}ms`);

// Concurrent TCP echo server (from script/tcp-echo.rhai)
let l = tcp_listen("127.0.0.1:9000");
loop {
    let conn = tcp_accept(l);
    thread_spawn(|c| {
        let peer = tcp_peer_addr(c);
        let line = tcp_read_line(c, 5000);
        tcp_write(c, `echo: ${line}` + "\n");
        tcp_close(c);
    }, conn);
}

// UDP listener (from script/udp-listen.rhai)
let s = udp_bind("127.0.0.1:9001");
loop {
    let r = udp_recv_from(s, 65536, 30000);
    print(`${r.addr} → ${r.data.len()} bytes`);
}

// UDP beacon sender
let s = udp_bind("0.0.0.0:0");
for i in 0..5 {
    udp_send_to(s, `beacon-${i}`, "127.0.0.1:9001");
    sleep_ms(200);
}
udp_close(s);
```

Threading bindings

0.56.0. Requires rhai's `sync` feature (enabled). `spawn` alone is reserved by Rhai — use `thread_spawn`.

Function	Description
`thread_spawn(fn_ptr)`	Spawn a closure. Returns a ThreadHandle.
`thread_spawn(fn_ptr, arg)`	With one forwarded arg.
`thread_spawn(fn_ptr, args_array)`	With N args.
`join(h)`	Block; returns the closure's return value (or the worker's error).

Function	Description
`tid()`	Current thread id (stable within a run).
`sleep(ms)`	Alias of `sleep_ms`.
`channel()`	Unbounded MPSC — returns `[sender, receiver]`.
`channel_bounded(n)`	Bounded; `try_send` returns false when full.
`send(tx, val)`	Blocking send.
`try_send(tx, val)`	Non-blocking; false on full.
`recv(rx)`	Blocking.
`recv(rx, timeout_ms)`	With timeout.
`try_recv(rx)`	Non-blocking; `()` when empty.

Examples

```
// Fan out probes, gather via channel
let c = channel();
let tx = c[0];
let rx = c[1];

let urls = [
    "https://a.example.com/",
    "https://b.example.com/",
    "https://c.example.com/",
];

for url in urls {
    thread_spawn(|u| {
        let r = http(u, #{ timeout_ms: 5000 });
        send(tx, `${u} → ${r.status}`);
    }, url);
}

for i in 0..urls.len() {
    print(recv(rx, 10000));
}

// Bounded channel — producer/consumer with back-pressure
let c = channel_bounded(4);
let tx = c[0];
let rx = c[1];

thread_spawn(|| {
    for i in 0..100 { send(tx, i); }
});

let sum = 0;
for j in 0..100 {
    sum += recv(rx, 1000);
}
print(`total: ${sum}`);

// Join for return value
let h = thread_spawn(|| {
    // Do something heavy
    let r = http("https://slow.example.com/");
    r.status
});
let status = join(h);
print(`completed with status ${status}`);
```

DNS binding

Function	Description
`dns(host)`	Default bundle: A, AAAA, MX, TXT, NS, CNAME, SOA. Returns a Map keyed by record type.
`dns(host, types_str)`	Custom types: `"A,AAAA,MX"`.
`dns(host, types_str, opts)`	`opts` may contain `servers: "1.1.1.1,8.8.8.8"`.

Examples

```
let r = dns("example.com");
print(`A: ${r.A.join(", ")}`);
print(`MX: ${r.MX.join(", ")}`);

// Custom resolver
let r = dns("example.com", "A,AAAA", #{ servers: "127.0.0.1:5353" });
print(r);

// Check DNSSEC chain presence
let r = dns("example.com", "DNSKEY,DS");
if r.DNSKEY.len() == 0 { print("no DNSKEY"); }
```

TLS probe binding

Function	Description
`tls(host [, opts])`	Handshake + cert introspection. Returns a Map with `subject`, `issuer`, `sans`, `not_before`, `not_after`, `days_remaining`, `fingerprints`, `chain`, `tls_version`, `cipher_suite`.

Examples

```
let t = tls("example.com:443");
print(`CN: ${t.subject}`);
print(`issuer: ${t.issuer}`);
print(`expires in ${t.days_remaining} days`);
if t.days_remaining < 14 { eprint("cert expiring soon"); return 1; }

// Verify an expected subject
let t = tls("api.example.com:443");
assert(t.sans.contains("api.example.com"), "SAN mismatch");

// Per-host SNI override
let t = tls("alt.example.com:443", #{ sni: "primary.example.com" });
```

Ping binding

Function	Description
`ping(host)`	Default TCP ping, 4 packets. Returns `#{ sent, received, loss_pct, min_ms, avg_ms, max_ms, …}`.
`ping(host, opts)`	`opts` supports `count`, `icmp: true`, `timeout_ms`.

Examples

```
let r = ping("8.8.8.8");
print(`${r.received}/${r.sent} received, avg ${r.avg_ms}ms`);

// ICMP (needs privileges on macOS/Linux for non-DGRAM types)
let r = ping("example.com", #{ icmp: true, count: 10 });

// TCP ping to a specific port
let r = ping("example.com:443");
assert(r.received > 0, "no reachability");
```

File transfer bindings

FTP

Function	Description
`ftp(url)`	FTP / FTPS anonymous listing or retrieve.
`ftp(url, opts)`	`opts`: `user`, `pass`, `ftps_implicit`.

SFTP

Function	Description
`sftp(url)`	SSH-backed listing or retrieve.
`sftp(url, opts)`	`opts` : `user` , `pass` , `privkey` , `port` .

TFTP

Function	Description
`tftp(url)`	RFC 1350 download.
`tftp(url, opts)`	`opts` : `blksize` (RFC 2348).

Gopher

Function	Description
`gopher(url)`	RFC 1436 selector fetch.

Examples

```
let listing = ftp("ftp://ftp.example.com/");
print(`${listing.entries.len()} entries`);

let data = ftp("ftp://ftp.example.com/file.bin");
print(`${data.size} bytes`);

let h = sftp("sftp://me@example.com/srv/app.log", #{ privkey:
"~/.ssh/ci_rsa" });
file_write_all("/tmp/app.log", h.body);

let img = tftp("tftp://tftp.example.com/boot.img", #{ blksize: 8192 });
print(img.size);

let g = gopher("gopher://gopher.floodgap.com/0/gopher/proxy");
print_raw(g.body);
```

Mail retrieval bindings

POP3

Function	Description
`pop3(url)`	Capability + message listing.
`pop3(url, opts)`	`opts`: `stls`, `retrieve_index` (int, 1-based).

IMAP

Function	Description
`imap(url)`	EXAMINE + capabilities.
`imap(url, opts)`	`opts`: `fetch` (int — message index), `peek: true`.

Examples

```
// POP3 probe with STLS
let r = pop3("pop3://alice:s3cr3t@pop.example.com/", #{ stls: true });
print(`${r.message_count} messages`);

let first = pop3("pop3s://alice:s3cr3t@pop.example.com/", #{
retrieve_index: 1 });
print_raw(first.body);

// IMAP peek
let r = imap("imaps://alice:s3cr3t@imap.example.com/INBOX", #{ fetch: 3,
peek: true });
print_raw(r.body);
```

SMTP binding

Function	Description
`smtp(url)`	Capability + STARTTLS probe.
`smtp(url, opts)`	Send a message. `opts` keys: `from`, `to` (array), `subject`, `body`, `headers`, `auth` (`"user:pass"`), `helo`, `no_starttls`, `dkim_key`, `dkim_selector`, `dkim_domain`.

Examples

```
// Probe only
let r = smtp("smtp://mail.example.com:25");
print(`capabilities: ${r.capabilities.join(", ")}`);

// Send
smtp("smtp://smtp.example.com:587", #{
    from: "me@example.com",
    to: ["you@example.com"],
    subject: "Automated notice",
    body: "Probe result: OK",
    auth: env("SMTP_CREDS"),
});

// DKIM-signed
smtp("smtp://smtp.example.com:587", #{
    from: "me@example.com",
    to: ["you@example.com"],
    subject: "Signed",
    body: "This message is DKIM-signed.",
    dkim_key: "~/.dkim/default.pem",
    dkim_selector: "default",
});
```

WebSocket binding

Function	Description
`ws(url)`	Handshake + Ping/Pong. Returns `#{ status, duration_ms, negotiated_protocol }`.
`ws(url, opts)`	`opts`: `headers`, `subprotocols`, `send_text`, `send_binary`.

Examples

```
let r = ws("wss://echo.websocket.events");
print(`connected in ${r.duration_ms}ms`);

// Send a frame, check response
let r = ws("wss://echo.websocket.events", #{ send_text: "hello" });
print(r.received);
```

Other protocol bindings

Brief interface reference; each has at least one example in `script/*.rhai`.

LDAP

```
let r = ldap("ldap://ldap.example.com");        // anonymous RootDSE query
```

RTSP

```
let r = rtsp("rtsp://camera.example.com/stream1");
```

Dict

```
let r = dict("dict://dict.org/d:recon");
```

NTP

```
let r = ntp("ntp://pool.ntp.org");
print(`offset: ${r.offset_ms}ms, rtt: ${r.rtt_ms}ms`);
```

Memcached

```
let r = memcached("memcache://cache.example.com/?version");
let s = memcached("memcache://cache.example.com/", #{ command: "stats" });
```

Redis

```
let r = redis("redis://cache.example.com/");      // PING
let info = redis("redis://cache.example.com/", #{ command: "INFO server"
});
```

MQTT

```
let r = mqtt("mqtt://broker.example.com", #{
    topic: "sensors/room-1",
    message: '{"temp": 21.3}',
});
```

Encoding & decoding bindings

0.55.0 expanded encode with Aztec + PDF417 and added decode.

Function	Description
`encode::qr(text)`	PNG Blob.
`encode::datamatrix(text)`	PNG Blob.
`encode::barcode(format, text)`	PNG Blob for any 1D/2D.
`encode::encode(format, text)`	PNG Blob (default renderer).
`encode::encode(format, text, output_format)`	output_format = `ascii` / `svg` / `png`. Returns string for ascii/svg, Blob for png.
`encode::encode(format, text, output_format, opts)`	opts: `qr_level: "L"|"M"|"Q"|"H"`.
`encode::decode(blob)`	Scan an image Blob → `#{ text, format }`.
`encode::list()`	Supported format names.

Examples

```
// Generate QR
let png = encode::qr("https://example.com");
file_write_all("/tmp/site.png", png);

// Tune QR durability
let png = encode::encode("qr", "sticker-text", "png", #{ qr_level: "H" });

// Get SVG
let svg = encode::encode("qr", "recon", "svg");
file_write_all("/tmp/site.svg", svg);

// Decode
let data = file_read("/tmp/site.png");
let r = encode::decode(data);
print(`${r.format}: ${r.text}`);

// Round-trip test
let png = encode::qr("hello");
let r = encode::decode(png);
assert(r.text == "hello", "round-trip failed");
```

Hashing binding

Ten algorithms plus CRC32.

Function	Description
`md5(data)`	MD5 (hex string).
`sha1(data)`	SHA-1.
`sha224` / `sha256` / `sha384` / `sha512`	SHA-2 family.
`sha3_256` / `sha3_512`	SHA-3.
`blake3(data)`	BLAKE3.
`crc32(data)`	CRC32 (hex, 8 chars).
`hmac_sha256(key, data)`	Keyed HMAC.

`data` accepts string or Blob.

Examples

```
let h = sha256(file_read("big.iso"));
print(h);

// HMAC
let sig = hmac_sha256("sharedsecret", "payload");
print(sig);

// Hash an HTTP body
let r = http("https://example.com/");
let fp = blake3(r.body);
print(`fingerprint: ${fp}`);
```

Compression & archive bindings

Compression

Function	Description
`compression::compress(algo, data [, level])`	algo = `gzip`, `deflate`, `brotli`, `zstd`, `bzip2`, `lz4`, `xz`, `snap`.
`compression::decompress(algo, data)`	Reverse.
`compression::decompress_auto(data)`	Auto-detect from magic bytes.
`compression::list()`	Supported algos.

Archive

Function	Description
`archive::create(dest_path, sources_array)`	Infer format from extension.
`archive::extract(src_path, dest_dir)`	Extract zip/tar/tar.gz/tar.xz/tar.bz2.

Examples

```
// Round-trip
let body = http("https://example.com/page.html").body;
let gz = compression::compress("gzip", body, 6);
print(`${body.len()} → ${gz.len()} bytes`);
let back = compression::decompress("gzip", gz);
assert(back.len() == body.len(), "round-trip");

// Zip up
archive::create("/tmp/backup.zip", ["file1.txt", "dir1/"]);
archive::extract("/tmp/backup.zip", "/tmp/restore/");
```

Encryption bindings

age + PGP shell-out.

Function	Description
`encrypt::keygen()`	New X25519 age keypair. Returns `#{ public, private }`.
`encrypt::encrypt(data, recipients)`	recipients = array of age-pub strings.
`encrypt::decrypt(data, identity)`	identity = secret key string.
`encrypt::encrypt_passphrase(data, passphrase)`	Scrypt-based.
`encrypt::decrypt_passphrase(data, passphrase)`	—

Examples

```
let kp = encrypt::keygen();
print(`public: ${kp.public}`);
file_write_all("~/.age/identity", kp.private);

let plaintext = "highly classified";
let ct = encrypt::encrypt(plaintext, [kp.public]);
let pt = encrypt::decrypt(ct, kp.private);
assert(pt.to_string() == plaintext, "round-trip");

let ct2 = encrypt::encrypt_passphrase("hello", "correct horse battery
staple");
```

Check-digit binding

Function	Description
`checkdigit::verify(algo, input)`	true / false.
`checkdigit::create(algo, partial)`	Computed full value.
`checkdigit::list()`	All 60+ algorithm names.

Examples

```
assert(checkdigit::verify("isbn13", "9780131103627"), "valid ISBN");

let full = checkdigit::create("ean13", "400638133393");
print(full);        // 4006381333931

for algo in checkdigit::list() {
    print(algo);
}
```

Sample-data binding

Function	Description
`sample::get(name)`	String content of a named sample.
`sample::list()`	Array of sample names.

Examples

```
let u = sample::get("json-user");
print(u);

for s in sample::list() { print(s); }
```

JWT binding

Function	Description
`jwt::view(token)`	Decode header + claims (no signature check).
`jwt::sign(payload_map, opts)`	opts: `alg`, `secret` or `key` path.
`jwt::validate(token, opts)`	opts: `alg`, `secret` or `key` / `pubkey`.

Examples

```
let t = jwt::sign(#{ sub: "alice", exp: now() + 3600 }, #{
    alg: "HS256",
    secret: env("JWT_SECRET"),
});

let ok = jwt::validate(t, #{ alg: "HS256", secret: env("JWT_SECRET") });
assert(ok, "validation failed");

let parts = jwt::view(t);
print(`alg: ${parts.header.alg}, sub: ${parts.payload.sub}`);
```

Email-protection binding

Function	Description
`email::spf(domain)`	SPF record + validation.
`email::dmarc(domain)`	DMARC policy.
`email::dkim(domain, selector)`	DKIM record for selector.
`email::mta_sts(domain)`	MTA-STS policy.
`email::bimi(domain [, selector])`	BIMI record.
`email::tls_rpt(domain)`	SMTP TLS-RPT record.

Examples

```
let s = email::spf("example.com");
if s.valid { print("SPF OK"); } else { print(`SPF: ${s.error}`); }

let d = email::dmarc("example.com");
print(`policy: ${d.policy}`);

let dk = email::dkim("example.com", "selector1");
print(dk.record);
```

Netstatus binding

Function	Description
`netstatus::run()`	Execute the probe bundle defined in `~/.recon/config.toml` `[netstatus]`.

```
let r = netstatus::run();
for p in r.probes {
    print(`${p.name}: ${p.status}`);
}
```

Text-encoding binding

Function	Description
`text::decode(blob, charset)`	Decode bytes → string.
`text::encode(string, charset)`	Encode string → bytes.
`text::detect(blob)`	Auto-detect (`chardetng`). Returns charset label.
`text::normalize_newlines(s, style)`	style = `unix` / `windows` / `mac` .
`text::list_charsets()`	Supported labels.

Examples

```
let bytes = file_read("greek.txt");
let charset = text::detect(bytes);
let utf8 = text::decode(bytes, charset);
file_write_all("greek-utf8.txt", text::encode(utf8, "utf-8"));

let crlf = text::normalize_newlines("line1\nline2\n", "windows");
```

String helpers

PHP-style free functions for string manipulation. All are top-level callables — `trim(s)` reads the same as in PHP — and co-exist with Rhai's existing String methods (which keep working).

Function	Description
`trim(s)` / `trim(s, mask)`	Strip whitespace, or any character in `mask`, from both ends.
`ltrim(s)` / `ltrim(s, mask)`	Strip from the left end only.
`rtrim(s)` / `rtrim(s, mask)`	Strip from the right end only.
`strrev(s)`	Reverse a string by Unicode codepoints (accented letters and emoji stay intact).
`strip_html(s)`	Remove every `<...>` segment. Quoted attribute values are respected; HTML entities pass through.
`nl2br(s)`	Insert ` ` before each `\n`, `\r\n`, or `\r`. HTML5 form (no trailing slash). Preserves the original newline.
`br2nl(s)`	Replace ` ` / ` ` / ` ` (any case, any inner whitespace) with `\n`. If the tag is immediately followed by an EOL, that EOL is kept — so `nl2br` ↔ `br2nl` round-trips.
`preg_match(pattern, subject)`	Returns an Array of capture strings: index 0 is the whole match, 1+ are groups. Empty array if no match.
`preg_replace(pattern, replacement, subject)`	Replace every match. `$1` / `${name}` in `replacement` expand to captures.
`arr.join(sep)` / `join(arr, sep)`	Concatenate an Array's elements with `sep` between them. Non-string elements are stringified via `to_string`.
`sprintf(fmt)` / `sprintf(fmt, arg)` / `sprintf(fmt, [a, b, …])`	Format and return a String.
`printf(fmt)` / `printf(fmt, arg)` / `printf(fmt, [a, b, …])`	Format and write to stdout. Returns the byte count.

Regex patterns accept either raw form (`"foo\\d+"`) or PHP-style delimited form (`"/foo\\d+/i"`) with the `i` / `m` / `s` / `x` flags.

`printf` / `sprintf` specifiers: `d` `i` `u` `o` `x` `X` `b` `f` `e` `E` `g` `G` `s` `c` `%`. Flags: `-` (left-align), `0` (zero-pad), `+` (force sign), space (space-sign), `#` (alt form for `o` / `x` / `X` / `b`). Width and precision are both supported. Rhai has no variadic concept; pass `[a, b, c]` for multi-arg formats.

Examples

```
print(trim("   hello   "));             // "hello"
print(ltrim("...path", "."));           // "path"
print(rtrim("file.log", ".log"));       // "file"

print(strrev("café"));                  // "éfac"
print(strip_html("<p>plain <b>text</b></p>"));   // "plain text"
print(nl2br("a\nb"));                   // "a<br>\nb"

let caps = preg_match("/^Host:\\s*(.+)$/i", "Host: example.com");
print(caps);                            // ["Host: example.com",
"example.com"]
print(preg_replace("\\s+", "-", "a  b   c"));     // "a-b-c"

print(["a", "b", "c"].join(", "));                // "a, b, c"
print(sprintf("%-10s %5d", ["alpha", 42]));       // "alpha           42"
print(sprintf("hex=%#x bin=%08b", [255, 10]));    // "hex=0xff
bin=00001010"
printf("pi=%.4f\n", 3.14159265);                  // writes "pi=3.1416\n"
```

Whois binding

Function	Description
`whois(domain)`	Two-hop whois (registry then registrar). Returns raw text.

```
let w = whois("example.com");
print_raw(w);
```

IPFS binding

Function	Description
`ipfs(url)`	Fetch `ipfs://` or `ipns://` via the configured gateway.

```
let data = ipfs("ipfs://QmYwAPJzv5CZsnA625s3Xf2nemtYgPpHdWEz79ojWnPbdG");
print(data.body.len());
```

Agent-browser binding

Exposed as a static module. Requires `agent-browser` on `$PATH`.

Function	Description
`agentBrowser::available`	Bool.
`agentBrowser::version`	Version string.
`agentBrowser::open(url)` / `open(url, opts)`	Navigate to URL. `opts` overrides global options for this call (0.75.0).
`agentBrowser::close()` / `close_all()`	End the current session / close every session.
`agentBrowser::back()` / `forward()` / `reload()`	Browser history navigation.
`agentBrowser::click(selector)` / `dblclick(selector)`	Click / double-click. Selector is CSS, XPath, or `@ref`.
`agentBrowser::hover(selector)` / `focus(selector)`	Hover / focus an element.
`agentBrowser::check(selector)` / `uncheck(selector)`	Toggle a checkbox.
`agentBrowser::fill(selector, text)`	Clear and fill a form field.
`agentBrowser::type_text(selector, text)`	Type into a field (per-character events). Named `type_text` because `type` is reserved in Rhai.
`agentBrowser::keyboard_type(text)` / `keyboard_insert(text)`	Keyboard-level typing into the focused element.
`agentBrowser::press(key)`	Key press (e.g. `"Enter"`, `"Tab"`, `"Control+a"`).
`agentBrowser::scroll(dir)` / `scroll(dir, px)`	Scroll the page.
`agentBrowser::scrollintoview(selector)`	Scroll an element into the viewport.
`agentBrowser::wait(selector_or_ms)`	Wait for element or a millisecond duration (string or int).
`agentBrowser::screenshot()` / `screenshot(path)` / `screenshot(path, opts)`	Save PNG (default path or explicit).
`agentBrowser::pdf(path)` / `pdf(path, opts)`	Save PDF.
`agentBrowser::snapshot()` / `snapshot(interactive_bool)` /	Accessibility-tree dump. `interactive=true` filters to

Function	Description
`snapshot(opts)` / `snapshot(interactive, opts)`	interactive elements only.
`agentBrowser::eval_js(js)` / `eval_js(js, opts)`	Execute JS, return the parsed value. Named `eval_js` because `eval` is reserved in Rhai.
`agentBrowser::get(what)` / `get(what, selector)`	Read page or element data. `what` ∈ `text` / `html` / `value` / `attr <name>` / `title` / `url` / `count` / `box` / `styles` / `cdp-url`. Returns a JSON object whose key matches `what`.
`agentBrowser::find(locator, value, action)` / `find(locator, value, action, text)`	Semantic-locator find + act. `locator` ∈ `role` / `text` / `label` / `placeholder` / `alt` / `title` / `testid` / `first` / `last` / `nth`. `action` ∈ `click` / `dblclick` / `hover` / `focus` / `fill` / `type` / `check` / `uncheck`. The 4-arg form passes `text` for fill/type.
`agentBrowser::is_visible(selector)` / `is_enabled(selector)` / `is_checked(selector)`	Predicate checks. Returns `bool`. Errors if no element matches — use `get("count", sel)` for an existence check that doesn't raise.
`agentBrowser::cmd(args_array)`	Raw passthrough to the CLI. Escape hatch for any subcommand without a typed wrapper (cookies, storage, tabs, network, mouse, etc.).

Existence check

To test whether a selector matches anything without raising an error:

```
fn exists(sel) {
    agentBrowser::get("count", sel).count > 0
}

if exists("#login-form") {
    agentBrowser::fill("#user", "alice");
}
```

`is_visible` / `is_enabled` / `is_checked` raise a Rhai error when no element matches, which aborts the script unless wrapped in `try { ... } catch { ... }`. `get("count", sel)` always returns `{ count: N }`, so `count == 0` is the no-match signal.

Examples

```
if !agentBrowser::available { return 2; }

agentBrowser::open("https://example.com/login");
agentBrowser::fill("#user", "alice");
agentBrowser::fill("#pass", "s3cr3t");
agentBrowser::click("button[type=submit]");
agentBrowser::screenshot("/tmp/after-login.png");
agentBrowser::close();
```

See `script/agent-browser-find.rhai`, `script/agent-browser-interaction.rhai`, `script/agent-browser-inspect.rhai`, `script/agent-browser-navigation.rhai`, `script/agent-browser-pdf.rhai`, and `script/agent-browser-cmd.rhai` for focused demos of each part of the surface.

Global options (0.75.0)

agent-browser accepts ~25 global launch / security / session options (see `agent-browser --help`). All are exposed as opts-map keys via `agentBrowser::set_default_options(opts)` (module-level defaults that apply to every binding call) plus per-call overrides on launch verbs.

Rhai key	agent-browser flag	Type
`ignore_https_errors`	`--ignore-https-errors`	bool
`allow_file_access`	`--allow-file-access`	bool
`headed`	`--headed`	bool
`auto_connect`	`--auto-connect`	bool
`annotate`	`--annotate`	bool
`no_auto_dialog`	`--no-auto-dialog`	bool
`content_boundaries`	`--content-boundaries`	bool
`confirm_interactive`	`--confirm-interactive`	bool
`verbose` / `quiet` / `debug` / `json`	`--verbose` / `--quiet` / `--debug` / `--json`	bool
`session`	`--session`	string
`session_name`	`--session-name`	string

Rhai key	agent-browser flag	Type
`executable_path`	`--executable-path`	string
`user_agent`	`--user-agent`	string
`proxy`	`--proxy`	string
`proxy_bypass`	`--proxy-bypass`	string
`state`	`--state`	string
`profile`	`--profile`	string
`provider`	`--provider`	string
`device`	`--device`	string
`color_scheme`	`--color-scheme`	string
`engine`	`--engine`	string
`model`	`--model`	string
`config`	`--config`	string
`screenshot_dir`	`--screenshot-dir`	string
`screenshot_format`	`--screenshot-format`	string
`download_path`	`--download-path`	string
`allowed_domains`	`--allowed-domains`	string
`action_policy`	`--action-policy`	string
`confirm_actions`	`--confirm-actions`	string
`cdp`	`--cdp`	int
`screenshot_quality`	`--screenshot-quality`	int
`max_output`	`--max-output`	int
`extension`	`--extension` (repeatable)	string or array
`browser_args`	`--args` (repeatable)	string or array
`headers`	`--headers <json>`	string or map

Module functions:

- `agentBrowser::set_default_options(opts: Map)` — store the defaults.
- `agentBrowser::clear_default_options()` — reset to empty.
- `agentBrowser::default_options()` → `Map` — read current defaults.

Per-call opts are accepted on `open`, `screenshot`, `snapshot`, `pdf`, and `eval`. They concatenate after defaults; agent-browser's flag parser uses last-wins for repeated single-value flags, so per-call options override defaults.

```
agentBrowser::set_default_options(#{
    ignore_https_errors: true,
    user_agent: "Recon/0.75",
    proxy: "http://proxy:3128",
});

agentBrowser::open("https://self-signed.example");
agentBrowser::click("#login");

// Per-call override:
agentBrowser::open("https://other.example", #{ user_agent: "Other/1.0" });

// Headers as a Rhai map (auto-serialized to JSON):
agentBrowser::open("https://api.example", #{
    headers: #{ Authorization: "Bearer x" },
});
```

SQLite binding

Function	Description
`sqlite(spec)` / `sqlite(spec, mode)`	Open. spec = path, `":memory:"`, or an alias. mode = `ro` | `rw` (default) | `rwc`.
`.query(sql)` / `.query(sql, params)`	Array of Maps.
`.query_one(sql, params)`	First row as a Map (or `()`).
`.query_value(sql, params)`	Single scalar value.
`.exec(sql, params)`	Row count affected.

Examples

```
let db = sqlite(":memory:");
db.exec("CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT);");
db.exec("INSERT INTO users (name) VALUES (?)", ["alice"]);
db.exec("INSERT INTO users (name) VALUES (?)", ["bob"]);

let rows = db.query("SELECT id, name FROM users ORDER BY id");
for r in rows {
    print(`${r.id}: ${r.name}`);
}

let count = db.query_value("SELECT COUNT(*) FROM users");
print(`total: ${count}`);
```

Document-conversion bindings

0.58.0. comrak for HTML; agent-browser for PDF.

Function	Description
`md_to_html(md)` / `md_to_html(md, opts)`	Markdown (string or Blob) → HTML string.
`md_to_pdf(md, dest)` / `md_to_pdf(md, dest, opts)`	Markdown → PDF at `dest`. Needs agent-browser.
`html_to_pdf(html, dest)`	HTML → PDF at `dest`. Needs agent-browser.

Opts map

Key	Default	Description
`toc`	false	Inject linkable TOC.
`toc_depth`	3	Include H1..H`N`.
`toc_title`	`"Contents"`	—
`title`	document default	`<title>` + PDF metadata.
`css`	—	Additional CSS (appended after default).
`no_default_css`	false	Skip bundled default CSS.
`gfm`	false	GitHub-flavored extensions.

Examples

```
let md = file_read("README.md").to_string();

// md → html
let html = md_to_html(md, #{ toc: true, toc_depth: 3, gfm: true, title:
"README" });
file_write_all("/tmp/readme.html", html);

// md → pdf
md_to_pdf(md, "/tmp/readme.pdf", #{ toc: true, gfm: true, title: "README"
});

// html → pdf
let html = http("https://example.com/report.html").body.to_string();
html_to_pdf(html, "/tmp/report.pdf");

// Fully custom styling
md_to_pdf(md, "/tmp/styled.pdf", #{
    no_default_css: true,
    css: file_read("print.css").to_string(),
    toc: true,
    toc_title: "Table of Contents",
});
```

`pdf_export_page`

Signature	Returns	Description
`pdf_export_page(pdf, page)`	`Blob`	Render page as PNG bytes.
`pdf_export_page(pdf, page, opts)`	`Blob`	Render page; opts map drives format / viewport / scale / quality.
`pdf_export_page(pdf, page, dest)`	`()`	Write to dest path; format from extension.
`pdf_export_page(pdf, page, dest, opts)`	`()`	Write to dest with full options.

Opts map keys:

Key	Type	Default	Notes
`viewport`	string `"WxH"`	`"1024x1366"`	Chrome CSS-pixel viewport.
`scale`	int	`2`	Device scale factor.
`quality`	int 0–100	`90`	JPEG/WEBP quality.

Key	Type	Default	Notes
`format`	`"png" | "jpeg" | "webp"`	inferred from dest extension, else `png`	Explicit override.

Examples:

```
// Write PNG, defaults
pdf_export_page("report.pdf", 1, "/tmp/cover.png");

// Larger WEBP
pdf_export_page("report.pdf", 1, "/tmp/cover.webp", #{
    viewport: "1920x2715",
    scale: 2,
    quality: 80,
});

// Return bytes
let png_bytes = pdf_export_page("report.pdf", 1);
let jpeg_bytes = pdf_export_page("report.pdf", 1, #{ format: "jpeg",
quality: 70 });
```

Requires `pdftoppm` (poppler-utils) on PATH. Install via `brew install poppler` (macOS) or `apt install poppler-utils` (Debian/Ubuntu).

AI bindings (`ai::*`)

The `ai::*` namespace dispatches a prompt to one of several subprocess-driven agent CLIs. Built-in backends: `claude` (Anthropic Claude Code), `codex` (OpenAI Codex), `copilot` (GitHub Copilot CLI), `gemini` (Google Gemini CLI). A user-defined `cmd` backend covers anything else. An HTTP backend (Anthropic Messages, OpenAI Chat Completions) is reserved as a follow-up — the builder API is forward-compatible.

Function	Returns	Description
`ai::ask(prompt)`	string	One-shot: `request().prompt(p).send()`.
`ai::request()`	builder	Fresh `AiRequest` builder.

Builder methods on an `AiRequest`:

Method	Behaviour
`.backend(name)`	Select backend (`claude` / `codex` / `copilot` / `gemini` / config-defined).

Method	Behaviour
`.model(name)`	Pass-through to the backend's `--model` flag or equivalent.
`.system(s)`	System prompt; singleton.
`.context(s)`	Append a context block; multiple calls accumulate.
`.prompt(s)` / `.user(s)`	Set the current user turn (singleton).
`.assistant(s)`	Append a prior assistant turn (for multi-turn replay).
`.max_tokens(n)`, `.temperature(f)`	Hints; backend honours when available.
`.timeout(secs)`	Wall-clock kill switch; default 60.
`.send()`	Returns `string` — model's reply. Throws on failure.
`.send_full()`	Returns map: `.text` `.backend` `.model` `.duration_ms` `.exit_code`.

Backend selection

Three-layer precedence per `.send()`:

1. Per-request `.backend(name)` / `.model(name)` / `.timeout(secs)`.
2. Env vars: `RECON_AI_BACKEND`, `RECON_AI_MODEL`, `RECON_AI_TIMEOUT`.
3. `~/.recon/config.toml` `[ai]` section.

No PATH fallback. If nothing selects a backend the request errors.

Config file

```
[ai]
default_backend = "claude"
default_model   = "sonnet"
timeout_secs    = 60

[ai.backends.claude]
model = "claude-sonnet-4-5"

[ai.backends.my-llm]
cmd          = ["my-llm-cli", "--print"]
model_flag   = "--model"
system_flag  = "--system"
```

The `[ai.backends.<name>]` block is the escape hatch for new agent CLIs without recompiling — name it, give it argv, optionally name its model and system flags. Scripts then write `req.backend("my-llm").model("v1")`.

Example

```
let req = ai::request()
    .system("You are a concise TLS expert.")
    .context("Cert subject CN: example.com")
    .context("Issuer: Let's Encrypt R3")
    .prompt("Is this a typical commercial cert?")
    .timeout(30);
print(req.send());
```

Errors

All `.send()` failures throw a Rhai script error prefixed `ai:`:

Tag	When
`ai: backend not configured`	no env, no config, no `.backend()`
`ai: backend '<name>' not found`	unknown built-in / config entry
`ai: CLI exited with status <N>:\n<stderr tail>`	non-zero exit
`ai: timed out after <N>s`	subprocess killed by timeout
`ai: spawn failed: <io error>`	CLI not on PATH, etc.
`ai: empty response from backend`	exit-zero but no stdout
`ai: no user prompt — call .prompt()/.user() before .send()`	builder validation
`ai: cannot append assistant turn — last turn is already assistant`	builder validation

Part IV — Appendices

Exit codes

Curl-compatible exit codes for common cases:

Code	Meaning
0	Success.
1	Generic protocol error.
2	Source-load error (`--compare` , etc.).
3	Agent-browser / Chrome unavailable (PDF features).
6	DNS resolution failed.
7	Connection refused.
22	HTTP ≥ 400 with `-f` (or `--fail-with-body`).
28	Operation timeout (`--max-time`).
35	TLS handshake error.
47	Redirect limit exceeded.
52	Empty reply from server.
55	Send error.
56	Receive error.
60	Peer cert cannot be authenticated.

Script-mode exit codes:

- `return N` from the top-level script becomes the process exit code (mod 256).
- Unhandled script errors → exit 1 (or the last `ProtocolExitCode` tag).

Environment variables

Variable	Read by	Description
`HTTP_PROXY` / `http_proxy`	`--proxy`	Proxy for http:// URLs.

Variable	Read by	Description
`HTTPS_PROXY` / `https_proxy`	`--proxy`	Proxy for https:// URLs.
`ALL_PROXY` / `all_proxy`	`--proxy`	Fallback proxy.
`NO_PROXY` / `no_proxy`	`--noproxy`	Bypass list.
`SSL_CERT_FILE`	`--cacert`	Extra trust-root bundle.
`CURL_CA_BUNDLE`	`--cacert`	Same as SSL_CERT_FILE.
`RECON_NO_PAGER`	`--help` / `--examples`	Disable paging.
`RECON_IPFS_GATEWAY`	`--ipfs-gateway`	Default IPFS gateway.
`RECON_CONFIG`	config file	Override path to `config.toml`.
`AGENT_BROWSER_JSON`	—	When `1`, agent-browser returns JSON (set internally by recon).
`EDITOR`	`--editor`	Editor binary for response inspection.
`HOME`	various	Used to locate `~/.recon/`.
`PAGER`	`--help` / `--examples`	Override default `less -FRX`.

Configuration file

`~/.recon/config.toml` — commented skeleton created by `recon --init`.

Structure

```
# Default flag overrides
[defaults]
connect_timeout = 30
max_time = 60
user_agent = "recon/0.58.1 (auto)"

# Netstatus probe bundle
[netstatus]
probes = [
    { name = "dns", target = "8.8.8.8" },
    { name = "http", url = "https://example.com/" },
    { name = "tls", target = "example.com:443" },
]

# Per-host SNI → cert mapping for --serve-tls
[[serve_sni]]
host = "a.example.com"
cert = "/etc/certs/a.pem"
key = "/etc/certs/a.key"

[[serve_sni]]
host = "b.example.com"
cert = "/etc/certs/b.pem"
key = "/etc/certs/b.key"
```

~/.recon/ layout

Created by `recon --init`:

```
~/.recon/
├── config.toml              # Commented skeleton; overrideable via
RECON_CONFIG
├── script/                  # Bare-word scripts (`recon --script NAME`)
│   └── *.rhai
├── jars/                    # Persistent cookie jars
│   └── <session>.db
├── sni/                     # Per-host TLS cert + key pairs for --serve-
tls
│   ├── a.example.com/cert.pem
│   └── a.example.com/key.pem
└── hsts.txt                 # Optional HSTS cache (curl-compatible)
```

Glossary

- **Sticky session** — a `browser()` handle whose cookies + headers persist across calls. Pair with `use_persistent_session` for cross-invocation persistence.
- **Script defaults** — the frozen snapshot of CLI flags (`-H` , `-k` , `--connect-timeout` , …) that every script binding inherits when the caller doesn't override via an opts map.
- **Bare-word script** — one that can be invoked by name because it lives in `~/.recon/script/` , e.g. `recon --script http` → `~/.recon/script/http.rhai` .
- **Protocol exit code** — recon's internal scheme for passing an exit code through a panic-style error path from protocol bindings (so a failed SMTP probe can exit with curl's `CURLE_URL_MALFORMAT` , for instance).
- **agent-browser** — external CLI that wraps Chrome DevTools Protocol. Used for `--browser-screenshot` , `--md-to-pdf` , `--html-to-pdf` , and the `agentBrowser::*` script bindings.

End of manual. For the curated-example companion, run `recon --examples` . For topic-specific deep dives, run `recon --help <topic>` .